

TRABAJO PRÁCTICO
SISTEMA DE GESTIÓN DE LICENCIAS

Métodos Ágiles para el Diseño de Software

Integrantes

Aldana Herbel, Franco Ocampo, Lara Musuruana, Santiago Nicle, Victoria Meichtry

UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL DE SANTA FE

Ingeniería en Sistemas de Información

Rodrigo Ledesma

2025 - 1° cuatrimestre



Índice

OBJETIVO	1
METODOLOGÍA.....	1
TECNOLOGÍAS UTILIZADAS	1
HISTORIAS REFINADAS	2
ESTIMACIÓN DE ESFUERZO Y PRIORIDAD	14
DESCRIPCIÓN DE LOS SPRINTS	16
ALCANCE	17
PRODUCT BACKLOG.....	18
PLANIFICACIÓN - SPRINT 1	19
DIVISIÓN DE TAREAS	19
EJECUCIÓN	19
DAILY SCRUM.....	34
REVISIÓN DEL SPRINT	38
GRÁFICO BURNDOWN.....	39
PLANIFICACIÓN - SPRINT 2	40
DIVISIÓN DE TAREAS.....	40
EJECUCIÓN	41
DAILY SCRUM.....	79
REVISIÓN DEL SPRINT	84
GRÁFICO BURNDOWN.....	86
CONCLUSIONES	87



OBJETIVO

El presente proyecto tiene como objetivo desarrollar un sistema de información para la Municipalidad de Santa Fe que permita gestionar de manera eficiente los datos relacionados con las licencias de conducir, utilizando metodologías ágiles. Las funcionalidades principales incluyen la emisión de nuevas licencias, la gestión de copias (duplicados, triplicados, etc.), renovaciones, cambios de domicilio, así como la realización de consultas sobre titulares, licencias, grupo y factor sanguíneo, donación de órganos, entre otros.

METODOLOGÍA

Para el desarrollo del sistema adoptamos una combinación de Scrum y Kanban, con el fin de facilitar el trabajo colaborativo y brindar mayor visibilidad al avance del proyecto. Utilizamos Jira como herramienta de gestión ágil, donde registramos el product backlog, historias de usuario, tareas y sprints. Esta plataforma nos permitió organizar el trabajo tanto de manera presencial como remota, generar métricas relevantes como el burn down chart y visualizar el flujo de trabajo a través de un tablero Kanban, con seguimiento del progreso de las tareas en tiempo real.

TECNOLOGÍAS UTILIZADAS

- **Control de versiones:** Git, con repositorio centralizado en GitHub, facilitando el trabajo colaborativo y la integración continua.
- **Base de datos:** PostgreSQL, administrada mediante pgAdmin, para garantizar una gestión robusta y eficiente de los datos.
- **Desarrollo web:** el sistema fue implementado como una aplicación web utilizando Java con el framework Spring Boot, lo que permitió construir una solución moderna, escalable y alineada con las tecnologías actuales del desarrollo de software.

Este enfoque no solo permitió alcanzar los objetivos propuestos, sino también fomentar el aprendizaje y la consolidación de conocimientos en tecnologías modernas por parte de los integrantes del equipo.



HISTORIAS REFINADAS

Se desarrollaron las historias de usuario refinadas empleando la plantilla para historias brindada por la cátedra. Dichas historias se formularon en base a las necesidades del cliente. A continuación, se puede visualizar cada una de ellas:

Número: 1	Nombre: Emitir una licencia
Usuario: Usuario administrativo	
Modificación Historia: Versión 1	Iteración Asignada: Sprint 1
Prioridad en Negocio: Alta	Puntos Estimados: 8
Riesgo en Desarrollo: Medio	
Descripción: Como usuario administrativo, quiero emitir una licencia de conducir para un titular, para que pueda obtener su carnet de conducir correspondiente. La funcionalidad permitirá registrar la emisión de licencias de distintas clases (A, B, C, D, E, F, G), teniendo en cuenta: • Que se respeten las edades mínimas requeridas para cada clase. • Que si se trata de una licencia profesional (clases C, D, E), se verifique la tenencia previa de licencia clase B durante al menos un año y la edad máxima de 65 años para la primera emisión. • Que se registren observaciones del titular si las hubiera. • Que se registre el usuario administrativo que realiza la emisión y el trámite correspondiente.	
Observaciones: • La fecha de emisión deberá coincidir con la fecha actual del sistema y no podrá ser modificada manualmente. • La funcionalidad deberá ser extensible para interactuar con los cálculos de vigencia y costos en futuras historias.	



Número: 2	Nombre: Calcular vigencia de la licencia
Usuario: Usuario administrativo	
Modificación Historia: Versión 1	Iteración Asignada: Sprint 1
Prioridad en Negocio: Alta	Puntos Estimados: 3
Riesgo en Desarrollo: Bajo	
Descripción: Como usuario administrativo, quiero que el sistema calcule automáticamente la vigencia de la licencia de conducir al momento de su emisión, para que se cumplan las normativas vigentes de acuerdo a la edad del titular. La funcionalidad permitirá determinar el período de vigencia de la licencia según las siguientes reglas: • Menores de 21 años: 1 año la primera vez y 3 años las renovaciones siguientes. • Hasta 46 años: 5 años de vigencia. • Hasta 60 años: 4 años de vigencia. • Hasta 70 años: 3 años de vigencia. • Mayores de 70 años: 1 año de vigencia. Además: • La fecha de vencimiento deberá coincidir en día y mes con la fecha de nacimiento del titular. • La fecha de inicio de vigencia será siempre la fecha actual del sistema y no podrá ser modificada manualmente.	
Observaciones: • Es necesario considerar en el futuro la posibilidad de modificaciones de normativa que afecten estos criterios de cálculo. • El cálculo de vigencia se utilizará posteriormente para determinar el costo de la licencia (otra historia).	



Número: 3	Nombre: Calcular costo de la licencia																																		
Usuario: Usuario administrativo																																			
Modificación Historia: Versión 1	Iteración Asignada: Sprint 1																																		
Prioridad en Negocio: Media	Puntos Estimados: 3																																		
Riesgo en Desarrollo: Bajo																																			
Descripción: Como usuario administrativo, quiero que el sistema calcule automáticamente el costo de la licencia de conducir al momento de su emisión o renovación, para que se respete la tabla de valores establecida y evitar errores humanos en la determinación del precio final. El cálculo del costo dependerá de: - La clase de la licencia solicitada (A, B, C, E, G). - La vigencia calculada (1, 3, 4 o 5 años). El sistema deberá aplicar las siguientes tarifas: <table border="1" style="width: 100%; border-collapse: collapse; text-align: center;"> <thead> <tr> <th rowspan="2">Clase</th> <th colspan="4">Vigencia del permiso</th> </tr> <tr> <th>5 años</th> <th>4 años</th> <th>3 años</th> <th>1 año</th> </tr> </thead> <tbody> <tr> <td>A</td> <td>\$ 40</td> <td>\$ 30</td> <td>\$ 25</td> <td>\$ 20</td> </tr> <tr> <td>B</td> <td>\$ 40</td> <td>\$ 30</td> <td>\$ 25</td> <td>\$ 20</td> </tr> <tr> <td>C</td> <td>\$ 47</td> <td>\$ 35</td> <td>\$ 30</td> <td>\$ 23</td> </tr> <tr> <td>E</td> <td>\$ 59</td> <td>\$ 44</td> <td>\$ 39</td> <td>\$ 29</td> </tr> <tr> <td>G</td> <td>\$ 40</td> <td>\$ 30</td> <td>\$ 25</td> <td>\$ 20</td> </tr> </tbody> </table> A este valor se sumará un cargo administrativo fijo de \$8. El operador no podrá modificar manualmente los valores.		Clase	Vigencia del permiso				5 años	4 años	3 años	1 año	A	\$ 40	\$ 30	\$ 25	\$ 20	B	\$ 40	\$ 30	\$ 25	\$ 20	C	\$ 47	\$ 35	\$ 30	\$ 23	E	\$ 59	\$ 44	\$ 39	\$ 29	G	\$ 40	\$ 30	\$ 25	\$ 20
Clase	Vigencia del permiso																																		
	5 años	4 años	3 años	1 año																															
A	\$ 40	\$ 30	\$ 25	\$ 20																															
B	\$ 40	\$ 30	\$ 25	\$ 20																															
C	\$ 47	\$ 35	\$ 30	\$ 23																															
E	\$ 59	\$ 44	\$ 39	\$ 29																															
G	\$ 40	\$ 30	\$ 25	\$ 20																															
Observaciones: Si se detectan combinaciones inválidas (por ejemplo, clase sin vigencia definida), se debe mostrar un mensaje de error y no permitir continuar.																																			



Número: 4	Nombre: Imprimir licencia
Usuario: Usuario administrativo	
Modificación Historia: Versión 1	Iteración Asignada: Sprint 1
Prioridad en Negocio: Media	Puntos Estimados: 3
Riesgo en Desarrollo: Bajo	
Descripción: Como usuario administrativo, quiero imprimir la licencia de conducir una vez finalizado el proceso de emisión, para que el titular pueda recibir su documento oficial y proceder al pago del trámite. La funcionalidad debe: • Generar una representación imprimible de la licencia de conducir con todos los datos correspondientes al titular, clase, vigencia, fecha de emisión, y observaciones si las hubiera. • Emitir automáticamente un comprobante de pago que detalle los datos y el costo total del trámite. • Permitir enviar ambos documentos (licencia y comprobante) a una impresora conectada o exportarlos como archivo PDF para su posterior impresión.	
Observaciones: • Se recomienda dejar preparada una interfaz que permita personalizar el diseño o actualizar campos si en el futuro cambia la normativa o presentación del carnet.	



Número: 5	Nombre: Renovar licencia
Usuario: Usuario administrativo	
Modificación Historia: Versión 1	Iteración Asignada: Sprint 2
Prioridad en Negocio: Media	Puntos Estimados: 8
Riesgo en Desarrollo: Medio	
Descripción: Como usuario administrativo, quiero renovar una licencia de conducir existente, para que el titular pueda seguir utilizándola legalmente en caso de vencimiento o cambios de datos personales. El sistema debe permitir renovar una licencia por dos motivos: • Fecha de vigencia vencida. • Modificación de datos del titular (por ejemplo: cambio de domicilio, apellido, etc.). El costo de la renovación por modificación de datos debe ser igual al de la emisión original del carnet. La nueva licencia debe conservar el historial del titular y mostrar la nueva fecha de vigencia. El sistema debe registrar el usuario administrativo que realizó la renovación.	
Observaciones: • El sistema debe validar que la licencia anterior exista y esté asociada al titular. • Se debe mostrar advertencia si la licencia anterior no está vencida ni hubo cambios en los datos del titular asociado.	



Número: 6	Nombre: Emitir una copia de la licencia
Usuario: Usuario administrativo	
Modificación Historia: Versión 1	Iteración Asignada: Sprint 2
Prioridad en Negocio: Baja	Puntos Estimados: 3
Riesgo en Desarrollo: Bajo	
Descripción: Como usuario administrativo, quiero emitir una copia de una licencia de conducir previamente emitida, para que el titular pueda obtener un duplicado, triplicado o más en caso de extravío, robo o deterioro del carnet original. La funcionalidad debe: • Emitir una nueva copia utilizando exactamente los datos de la última licencia válida. • Registrar en el trámite el número de copia emitida (duplicado, triplicado, etc.). • Aplicar un costo fijo de \$50 por la emisión de copia. • Registrar la fecha del trámite y el usuario que lo realizó.	
Observaciones: • No se debe permitir la modificación de datos personales del titular durante este proceso. • El historial de copias emitidas debe mantenerse registrado para fines de auditoría.	



Número: 7	Nombre: Emitir listado de licencias que han expirado
Usuario: Usuario administrativo	
Modificación Historia: Versión 1	Iteración Asignada: Sprint 2
Prioridad en Negocio: Media	Puntos Estimados: 3
Riesgo en Desarrollo: Bajo	
Descripción: Como usuario administrativo, quiero obtener un listado de licencias de conducir que han expirado, para poder realizar acciones de contacto, seguimiento o renovación con los titulares. El listado debe incluir al menos los siguientes datos: • Nombre completo del titular. • Tipo y número de documento del titular. • Clase de licencia. • Fecha de vencimiento. • Estado actual (no vigente). El usuario debe poder filtrar por: • Rango de fechas de expiración. • Clase de licencia.	
Observaciones: • El listado debe excluir licencias que ya estén en proceso de renovación, para evitar duplicidad en el seguimiento y asegurar que el foco esté sólo en aquellas que requieren acción administrativa.	



Número: 8	Nombre: Emitir listado de licencias vigentes por criterios
Usuario: Usuario administrativo	
Modificación Historia: Versión 1	Iteración Asignada: Sprint 2
Prioridad en Negocio: Media	Puntos Estimados: 3
Riesgo en Desarrollo: Bajo	
Descripción: Como usuario administrativo, quiero generar listados filtrados de licencias vigentes, para que pueda consultar la información de los titulares activos según diferentes criterios útiles para la gestión y auditoría. Los criterios de filtrado serán: • Nombre y apellido del titular (búsqueda parcial o exacta). • Grupo sanguíneo y factor RH. • Donación de órganos (sí/no). El listado debe incluir: • Datos del titular (nombre y apellido, documento, fecha de nacimiento, dirección). • Clase de licencia y fecha de vencimiento. • Información relevante asociada al criterio de búsqueda. Las licencias no vigentes no deben incluirse en el listado, pero deben permanecer almacenadas para historial.	
Observaciones: • Esta funcionalidad puede servir como base para futuras ampliaciones (reportes estadísticos, cruces de datos con entidades externas, etc.). • El historial de licencias vencidas debe estar accesible en otra sección para no mezclar registros.	



Número: 9	Nombre: Dar de alta un titular
Usuario: Usuario administrativo	
Modificación Historia: Versión 1	Iteración Asignada: Sprint 1
Prioridad en Negocio: Alta	Puntos Estimados: 2
Riesgo en Desarrollo: Bajo	
Descripción: Como usuario administrativo, quiero dar de alta un nuevo titular de licencia de conducir, para que pueda iniciar el proceso de obtención de su licencia conforme a los requisitos del municipio. El sistema debe permitir registrar a una persona como titular si presenta documentación válida y completa. Los datos requeridos para el alta son: • Tipo y número de documento. • Apellido y nombre. • Fecha de nacimiento. • Dirección. • Grupo sanguíneo y factor RH. • Donante de órganos (sí/no). El sistema debe validar que: • Todos los campos obligatorios estén completos. • No exista ya un titular con el mismo tipo y número de documento.	
Observaciones: • Esta funcionalidad es necesaria para cualquier otro trámite posterior (emisión, renovación, etc.), ya que el titular debe existir en el sistema. • Se recomienda incorporar una verificación visual o por sistema para evitar errores de carga de datos críticos como el grupo sanguíneo o la clase solicitada.	



Número: 10	Nombre: Modificar datos de un titular existente
Usuario: Usuario administrativo	
Modificación Historia: Versión 1	Iteración Asignada: Sprint 2
Prioridad en Negocio: Baja	Puntos Estimados: 2
Riesgo en Desarrollo: Bajo	
Descripción: Como usuario administrativo, quiero modificar los datos personales de un titular previamente registrado, para que la información del sistema esté actualizada y refleje los cambios en los datos del contribuyente. La funcionalidad debe: ● Permitir buscar al titular por tipo y número de documento. ● Mostrar los datos actuales del titular en un formulario editable. ● Permitir modificar: apellido, nombre, fecha de nacimiento, dirección, grupo sanguíneo, factor RH y estado de donación de órganos. ● Validar los datos antes de guardar.	
Observaciones: ● Esta funcionalidad puede integrarse con la historia de renovación, ya que ciertos cambios pueden derivar en una actualización del carnet. ● Se recomienda incluir controles para evitar cambios innecesarios (por ejemplo, guardar sin haber modificado nada).	



Número: 11	Nombre: Dar de alta un usuario administrativo
Usuario: Usuario administrador (superusuario)	
Modificación Historia: Versión 1	Iteración Asignada: Sprint 2
Prioridad en Negocio: Baja	Puntos Estimados: 2
Riesgo en Desarrollo: Medio	
Descripción: Como administrador del sistema, quiero dar de alta nuevos usuarios administrativos, para que puedan operar el sistema y gestionar trámites vinculados a licencias de conducir. El sistema debe permitir al administrador crear usuarios administrativos con: • Nombre y apellido. • Tipo y número de documento. • Usuario de acceso (nombre de usuario único). • Contraseña. El sistema debe validar que no existan nombres de usuario duplicados.	
Observaciones: • El administrador debe tener acceso total para crear y gestionar usuarios, asegurando que solo personas con el rol adecuado puedan hacer cambios importantes en el sistema.	



Número: 12	Nombre: Modificar datos de un usuario administrativo
Usuario: Usuario administrador (superusuario)	
Modificación Historia: Versión 1	Iteración Asignada: Sprint 2
Prioridad en Negocio: Baja	Puntos Estimados: 2
Riesgo en Desarrollo: Bajo	
Descripción: Como usuario administrador del sistema, quiero modificar los datos de los usuarios administrativos registrados, para que pueda mantener la información del personal actualizada y realizar tareas de mantenimiento del sistema. La funcionalidad debe: ● Permitir buscar usuarios por nombre de usuario, tipo de documento y/o número de documento. ● Mostrar los datos actuales del usuario seleccionado. ● Permitir modificar los datos personales del usuario (nombre, apellido, contraseña). ● Validar que sólo un usuario con privilegios de administrador pueda realizar estas modificaciones.	
Observaciones: ● Esta historia se complementa con la funcionalidad de alta de usuarios y el control de roles en el sistema.	



Número: 13	Nombre: Iniciar sesión en el sistema
Usuario: Usuarios (administradores y administrativos)	
Modificación Historia: Versión 1	Iteración Asignada: Sprint 2
Prioridad en Negocio: Baja	Puntos Estimados: 3
Riesgo en Desarrollo: Bajo	
Descripción: Como usuario administrador/administrativo, quiero iniciar sesión en el sistema con mi nombre de usuario y contraseña, para acceder de forma segura a las funcionalidades disponibles según mi rol. ● El sistema debe permitir ingresar nombre de usuario y contraseña. ● Se deben validar las credenciales contra los datos almacenados. ● Si los datos son correctos, el sistema debe redirigir al panel principal para el rol correspondiente. ● Si los datos son incorrectos, debe mostrar un mensaje claro de error. ● El sistema debe mantener la sesión iniciada mientras el usuario navega. ● Debe existir una opción para cerrar sesión.	
Observaciones: ● Se debe utilizar cifrado seguro para la contraseña (por ejemplo, hashing con BCrypt). ● Debe considerarse la posibilidad de agregar roles para restringir acceso a ciertas funciones.	

La historia N°13 (login) fue incorporada al backlog debido a que constituye un requisito fundamental de seguridad y control de acceso en cualquier sistema administrativo.

Dado que las funcionalidades principales del sistema (como emisión o renovación de licencias) están destinadas exclusivamente a usuarios autorizados, era necesario implementar un mecanismo que garantice que sólo usuarios administrativos registrados puedan operar el sistema. Además, el login permite registrar la trazabilidad de las acciones realizadas por cada usuario.

ESTIMACIÓN DE ESFUERZO Y PRIORIDAD

Para realizar la estimación de esfuerzo de cada una de las historias de usuario decidimos aplicar la técnica de planning póker, utilizando la aplicación web propuesta por la cátedra (planningpokeronline.com). Esta dinámica nos permitió obtener una visión compartida del trabajo a

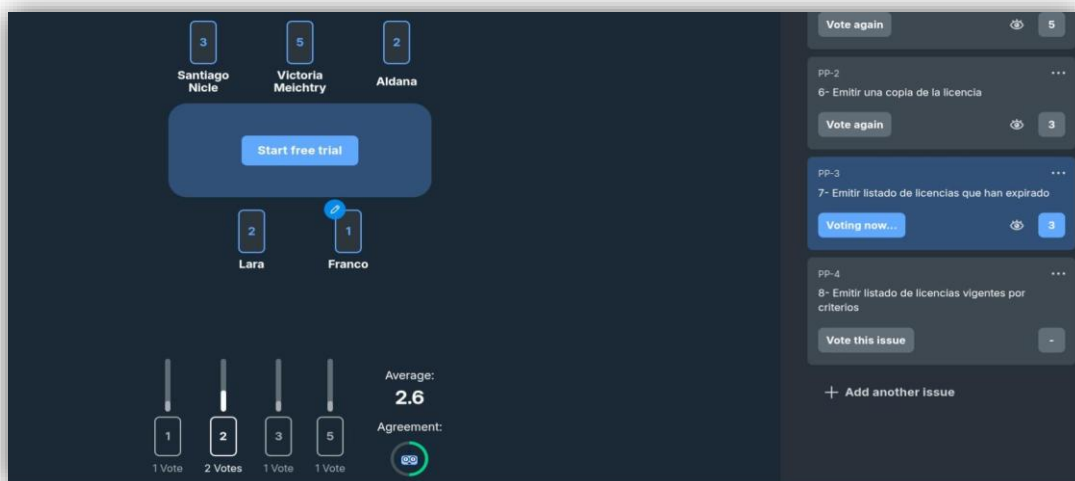


realizar, promoviendo el debate y el consenso entre los miembros del equipo. Algunos de los debates generados fueron:

- Historia N°1 (Emitir una licencia): algunos integrantes del equipo estimaban 5 puntos, mientras que otros consideraban que debían asignarse 8. El desacuerdo se centró en la cantidad de validaciones necesarias (edad mínima, clase profesional, licencias previas) y en la interacción con otras clases y funcionalidades.
Luego de una discusión técnica, se reconoció que esta historia era el núcleo del sistema y tenía múltiples dependencias lógicas, por lo que se consensuó una estimación de 8 puntos.
- Historia N°4 (Imprimir licencia): durante la estimación de esta historia, la mayoría del equipo coincidió en que la funcionalidad no implicaba una lógica compleja, pero sí presentaba desafíos técnicos específicos relacionados con el formato de salida. Las estimaciones iniciales oscilaron entre 2 y 3 puntos. El principal punto de discusión fue la generación del archivo PDF desde el navegador, ya que ningún integrante del equipo había implementado antes una solución similar y existía incertidumbre sobre qué herramientas o bibliotecas utilizar. Considerando este aspecto y ante la falta de experiencia previa en este tipo de tareas, decidimos actuar con prudencia, tomando la estimación más alta propuesta (3 puntos). Esto nos permitió contar con un margen adecuado frente a posibles complicaciones durante la implementación.
- Historia N°5 (Renovar licencia): esta historia recibió estimaciones que iban desde 5 hasta 8 puntos. Algunos miembros consideraban que la lógica era parecida a la de emisión de una licencia, pero otros advirtieron que implicaba validaciones adicionales (verificar cambios de datos, validar existencia de licencia previa, registrar renovación sin perder historial). Luego de discutir los riesgos y los casos límite (como renovaciones por cambios menores), se optó por tomar la estimación más alta (8 puntos), considerando que el margen de error era mayor y que esta historia tenía alta dependencia de otras funcionalidades.

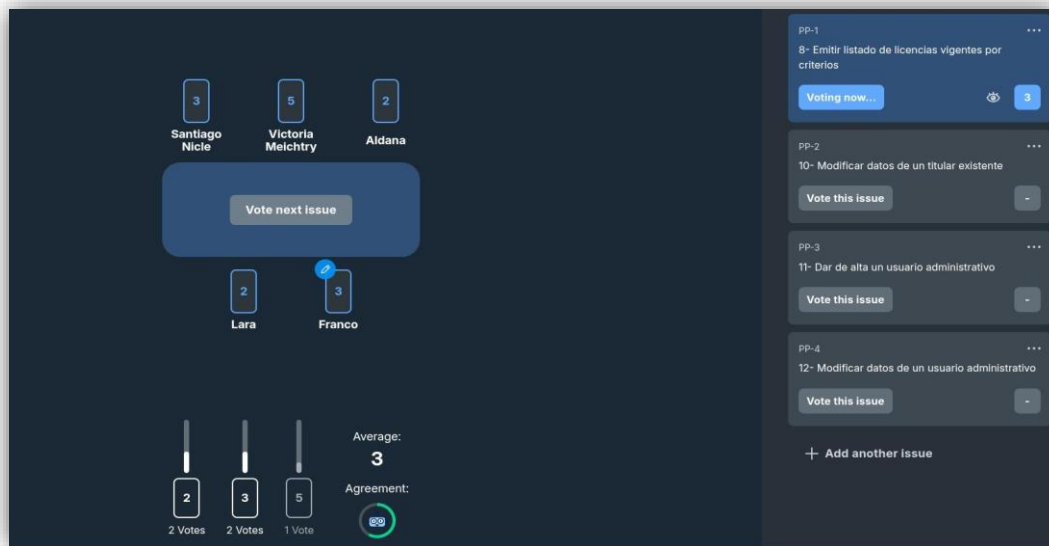
A continuación, se muestran otras de las votaciones realizadas con planning póker:

Historia N°7 (Emitir listado de licencias que han expirado):

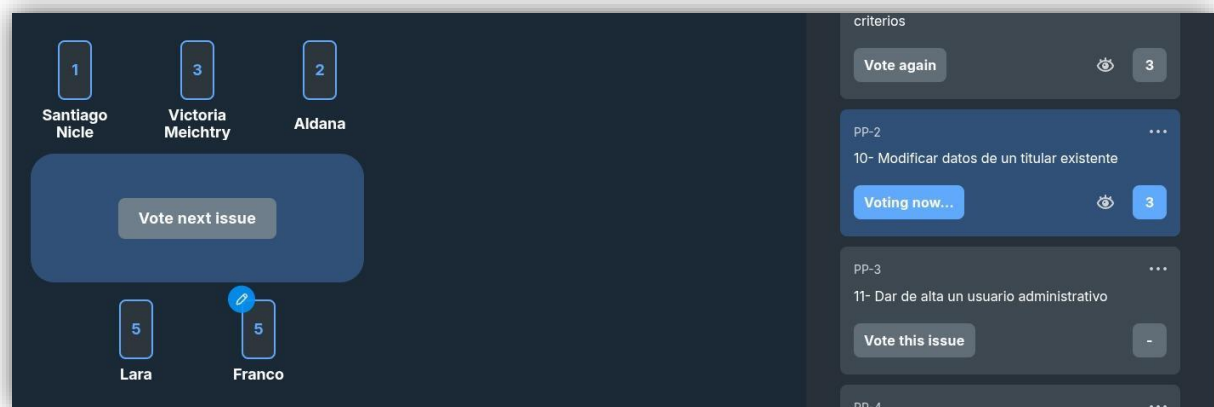




Historia N°8 (Emitir listado de licencias vigentes por criterios):



Historia N°10 (Modificar datos de un titular existente):



En cuanto a las prioridades de cada historia, éstas fueron definidas por el cliente. Sin embargo, a partir de un debate entre los miembros del equipo de desarrollo, se concluyó que era conveniente modificar algunas de las prioridades para lograr el correcto funcionamiento de la aplicación, puesto que algunas historias que se consideraban de baja prioridad son fundamentales para la ejecución de otras de prioridad media/alta. Estos cambios fueron consultados y validados por el cliente.

DESCRIPCIÓN DE LOS SPRINTS

DURACIÓN DEL SPRINT

Definimos un tamaño de sprint de 4 semanas (20 días hábiles) ya que recién estamos comenzando a trabajar con metodologías ágiles y consideramos que esta duración permite tener un mayor margen para planificar y desarrollar, además de reducir la presión inicial.



CANTIDAD DE SPRINTS: 2.

VELOCIDAD DEL EQUIPO

La velocidad representa la cantidad de puntos de historia que se estima que el equipo puede completar por sprint.

VELOCIDAD ESTIMADA

SPRINT 1: 27 story points.

SPRINT 2: 18 story points.

TOTAL ESTIMADO PARA EL PROYECTO

45 story points en 2 sprints.

ALCANCE

El alcance representa el conjunto de funcionalidades (historias de usuario) que nos comprometemos a entregar en el proyecto.

En el **SPRINT 1** se incluyen historias de prioridad alta y media. En este sprint, se planea realizar 5 historias de usuario.

En el **SPRINT 2**, se decidió desarrollar las historias de prioridad baja y las de prioridad media restantes. Este sprint abarca 8 historias de usuario.

De esta manera, se definió que el alcance total son todas las funcionalidades solicitadas por el cliente.



PRODUCT BACKLOG

Se desarrolló el product backlog en base a las historias de usuario previamente definidas. Se estableció un orden para estas historias según las prioridades aceptadas por el cliente. También se decidió qué historias desarrollar en cada sprint, según su tamaño de esfuerzo.

Identificador de la historia	Descripción	Estado	Esfuerzo	Sprint	Prioridad
9	Dar de alta un titular	Finalizado	2	1	Alta
1	Emitir una licencia	Finalizado	8	1	Alta
2	Calcular vigencia de licencia	Finalizado	3	1	Alta
3	Calcular costo de licencia	Finalizado	3	1	Media
5	Renovar licencia	Pendiente	8	1	Media
4	Imprimir licencia	Finalizado	3	1	Media
8	Emitir listado de licencias vigentes por criterios	Pendiente	3	2	Media
7	Emitir listado de licencias que han expirado	Pendiente	3	2	Media
6	Emitir una copia de la licencia	Pendiente	3	2	Baja
10	Modificar datos de un titular existente	Pendiente	2	2	Baja
11	Dar de alta un usuario administrativo	Pendiente	2	2	Baja
13	Iniciar sesión en el sistema	Pendiente	3	2	Baja
12	Modificar datos de un usuario administrativo	Pendiente	2	2	Baja
			Esfuerzo Sprint 1	27	
			Esfuerzo Sprint 2	18	
			TOTAL	45	

Tabla 1 – Product backlog



PLANIFICACIÓN - SPRINT 1

La fecha de inicio de este sprint fue el 02/05/2025.

DIVISIÓN DE TAREAS

A continuación, se presenta una tabla con las tareas necesarias para cada historia, junto con el programador asignado y la cantidad de task points necesarios para su realización. Los posibles estados son: “Pendiente”, “En proceso” y “Finalizado”.

ID Historia	ID Tarea	Nombre tarea	Dueño	Esfuerzo	Estado
1	18	Diseñar interfaz de emisión de licencia.	Aldana	2	Finalizado
1	19	Desarrollar lógica de emisión de licencias y validación de datos.	Lara y Aldana	5	Finalizado
1	20	Realizar pruebas de emisión de licencias.	Victoria	3	Finalizado
1	34	Desarrollar endpoint "buscarTitular".	Franco	2	Finalizado
2	21	Desarrollar lógica de cálculo de vigencia de una licencia.	Aldana	2	Finalizado
3	22	Desarrollar lógica de cálculo de costo de una licencia.	Santiago	3	Finalizado
3	31	Desarrollar test de la lógica de cálculo de costo de una licencia.	Santiago	2	Finalizado
4	23	Diseñar las salidas a imprimir.	Aldana	2	Finalizado
4	24	Desarrollar lógica de imprimir licencia.	Franco	4	Finalizado
9	13	Codificar las clases del modelo y la base de datos.	Franco	4	Finalizado
9	14	Diseñar interfaz de menú titulares.	Franco	2	Finalizado
9	15	Diseñar interfaz de formulario de alta de titular con sus validaciones.	Aldana	3	Finalizado
9	16	Desarrollar lógica de alta de titular y validación de datos.	Santiago	2	Finalizado
9	17	Realizar prueba de registro de titulares.	Santiago	2	Finalizado
9	33	Refactorizar "registrarTitular" y "calcularCostoLicencia".	Santiago	2	Finalizado

Tabla 2 – División de tareas del sprint 1

EJECUCIÓN

En esta sección se describe con detalle cómo se implementaron las tareas, y se incluyen comentarios de los desarrolladores que justifican los motivos de las desviaciones respecto a las estimaciones realizadas. Además, se adjunta el código fuente principal de cada tarea y las interfaces de usuario correspondientes.

ID HISTORIA: 1 - ID TAREA: 18

Se desarrolló la interfaz donde se requieren los siguientes datos: titular de la licencia (tipo y número de documento), clase de licencia (ClaseLicencia), observaciones del titular (String).

Si el titular ya tiene una licencia vigente de la misma clase, no cumple con alguno/s de los requisitos para emitir la licencia, hay campos obligatorios vacíos o se ingresaron datos inválidos, se marcan los campos correspondientes y/o se muestran mensajes de error.

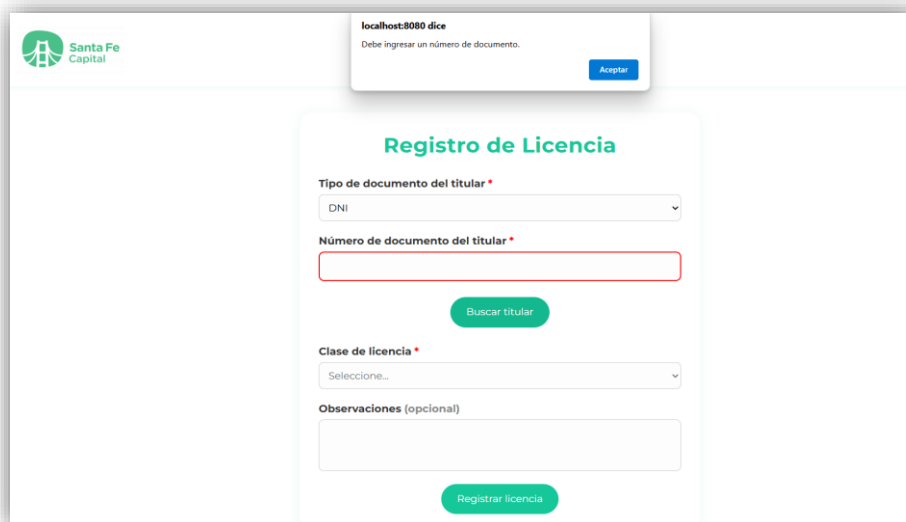
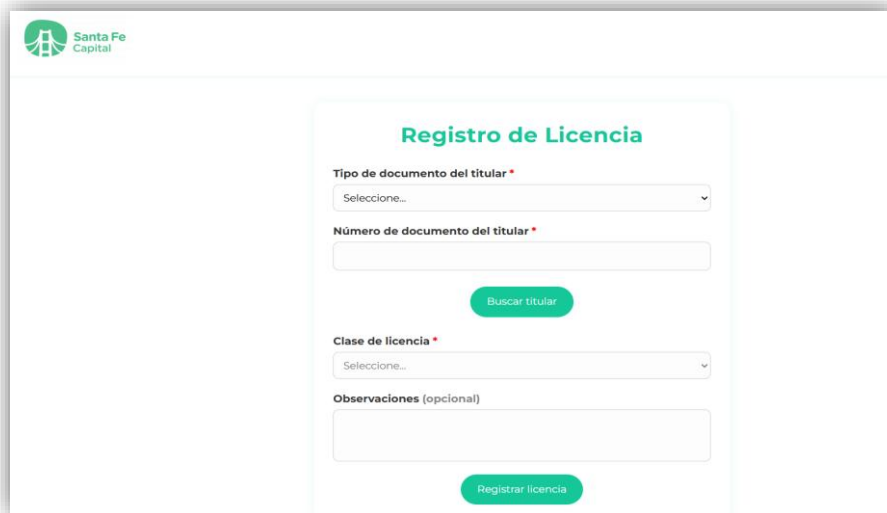
También se implementó el `LicenciaController` que se encarga de mostrar la interfaz y comunicarse con ella.

```
@Tag(name = "Licencia Controller", description = "Operaciones para la emisión de licencias")
@Controller
@RequestMapping("/api/licencias")
public class LicenciaController {

    private final LicenciaServiceImpl licenciaService;
    private final TitularServiceImpl titularService;
    private final TramiteServiceImpl tramiteService;

    @Autowired
    public LicenciaController(LicenciaServiceImpl licenciaService, TitularServiceImpl titularService, TramiteServiceImpl tramiteService) {
        this.licenciaService = licenciaService;
        this.titularService = titularService;
        this.tramiteService = tramiteService;
    }

    @GetMapping("/registroLicencia")
    public String mostrarFormulario() { return "emisionLicencia"; }
}
```



Comentario: Llevó el tiempo estimado.



ID HISTORIA: 1 - ID TAREA: 19

Se llevó a cabo la lógica que permite emitir una licencia de conducir para un titular. Esto incluye la validación de requisitos por clase de licencia (edad mínima, tenencia previa de clase B, límites por edad para licencias profesionales), la generación automática de la fecha de emisión y vigencia, el cálculo del costo correspondiente y el registro completo del trámite en la base de datos.

Se crearon la interfaz `LicenciaService` y la clase `LicenciaServiceImpl`.

Se agregaron los métodos `registrarLicencia(LicenciaRecord licenciaRecord)`, `mostrarComprobante(TipoDocumento tipoDocumento, String documento)` y `mostrarLicencia(TipoDocumento tipoDocumento, String documento)` a la clase `LicenciaController`. También se crearon las interfaces `TramiteRepository` y `TramiteService` (y luego la clase `TramiteServiceImpl`) para poder registrar el trámite correspondiente en la base de datos, y las interfaces `UsuarioRepository` y `UsuarioService` (y la clase `UsuarioServiceImpl`) para poder buscar el usuario administrativo correspondiente y registrarlo en el trámite (se creó un usuario manualmente en la base de datos y será utilizado hasta que se complete la historia relacionada con el alta de un usuario administrativo).

```
public Licencia emitirLicencia(Titular titular, String claseLicencia, String observaciones) throws ClaseEmisionInvalidaException {
    // Obtener las licencias del titular
    List<Licencia> licenciasTitular = titular.getLicencias();

    // Buscar si el titular tiene una licencia vigente del tipo solicitado
    Optional<Licencia> licenciaTipo = licenciasTitular.stream()
        .filter(licencia -> licencia.getClase().equalsIgnoreCase(claseLicencia))
        .findFirst();

    if (licenciaTipo.isPresent()) {
        Licencia l = licenciaTipo.get();
        if (!l.getEstaVigente()) {
            throw new ClaseEmisionInvalidaException("El titular ya posee una licencia de tipo " + claseLicencia + " que no está vigente. Debe renovarla.");
        } else if (l.getFechaVencimiento().before(java.sql.Date.valueOf(java.time.LocalDate.now()))) {
            l.setEstaVigente(false);
            repository.save(l);
            throw new ClaseEmisionInvalidaException("El titular ya posee una licencia de tipo " + claseLicencia + " que está vencida. Debe renovarla.");
        } else throw new ClaseEmisionInvalidaException("El titular ya posee una licencia vigente de tipo " + claseLicencia + ".");
    }

    else if (claseLicencia.equalsIgnoreCase("C") || claseLicencia.equalsIgnoreCase("D") || claseLicencia.equalsIgnoreCase("E")) {
        // Validar que el titular tenga más de 21 años
        if (titular.getEdad() < 21 && (claseLicencia.equalsIgnoreCase("C") || claseLicencia.equalsIgnoreCase("D") || claseLicencia.equalsIgnoreCase("E"))) {
            throw new ClaseEmisionInvalidaException(mensaje: "El titular debe tener al menos 21 años para obtener una licencia de tipo C, D o E.");
        }

        // Buscar si el titular tiene una licencia de tipo B
        boolean tieneLicenciaTipoB = licenciasTitular.stream()
            .anyMatch(licencia -> licencia.getClase().equalsIgnoreCase("B") &&
                licencia.getFechaInicio().before(java.sql.Date.valueOf(java.time.LocalDate.now().minusYears(yearsToSubtract:1))));

        // Validar que tenga una licencia tipo B con más de un año de antigüedad
        if (!tieneLicenciaTipoB) {
            throw new ClaseEmisionInvalidaException(mensaje: "El titular debe haber tenido una licencia tipo B con al menos un año de antigüedad para obtener una licencia de tipo C, D o E.");
        }

        // Validar que el titular tenga menos de 65 años
        if (titular.getEdad() >= 65 && (claseLicencia.equalsIgnoreCase("C") || claseLicencia.equalsIgnoreCase("D") || claseLicencia.equalsIgnoreCase("E"))) {
            throw new ClaseEmisionInvalidaException(mensaje: "El titular debe tener menos de 65 años para obtener una licencia de tipo C, D o E.");
        }

    } else if (titular.getEdad() < 17) {
        throw new ClaseEmisionInvalidaException(mensaje: "El titular debe tener al menos 17 años para obtener una licencia de tipo A, B, F o G.");
    }
}
```



```
// Emitir la licencia
Licencia nuevaLicencia = new Licencia();
nuevaLicencia.setTitular(titular);
nuevaLicencia.setClase(claseLicencia);
nuevaLicencia.setEstaVigente(estaVigente,true);
nuevaLicencia.setObservaciones(observaciones);
nuevaLicencia.setFechaInicio(java.sql.Date.valueOf(java.time.LocalDate.now()));

// Calcular años de vigencia según titular y clase de licencia
int añosVigencia = calcularVigenciaLicencia(titular, claseLicencia);

int añoVencimiento = LocalDate.now().getYear() + añosVigencia;

// Convertir fecha de nacimiento (Date) a LocalDate
LocalDate fechaNacimiento = ((java.sql.Date) titular.getFechaNacimiento()).toLocalDate();

// Crear la fecha de vencimiento con mismo mes y día que el nacimiento, pero en el año calculado
LocalDate fechaVencimiento = LocalDate.of(
    añoVencimiento,
    fechaNacimiento.getMonth(),
    fechaNacimiento.getDayOfMonth()
);

// Establecer la fecha de vencimiento en la licencia (conversión a java.sql.Date)
nuevaLicencia.setFechaVencimiento(java.sql.Date.valueOf(fechaVencimiento));

repository.save(nuevaLicencia);

//Registro del trámite
Trámite tramite = new Trámite();
tramite.setFecha(java.sql.Date.valueOf(java.time.LocalDate.now()));
tramite.setDescripcion("Emisión de licencia de conducir clase " + claseLicencia);
tramite.setCosto(calcularCostoLicencia(claseLicencia, añosVigencia));
tramite.setTitularAsociado(titular);
tramite.setUsuarioResponsable(usuarioService.buscarUsuarioPorId(idUsuario:1)); // POR AHORA HASTA QUE HAGAMOS EL OTRO SPRINT
tramite.setLicenciaAsociada(nuevaLicencia);
tramiteService.registrarTrámite(tramite);

return nuevaLicencia;
}
```

Comentario: Dado que hubo complicaciones en algunas partes de la tarea, colaboraron ambas programadoras para poder finalizarla. Llevó más tiempo de lo estimado (aproximadamente 2 horas más).

ID HISTORIA: 1 - ID TAREA: 34

Se desarrolló el endpoint buscarTitular(TipoDocumento tipoDocumento, String documento) en la clase TitularController. Además, se agregó la clase TitularMapper y se refactorizaron las clases para manejarse con mapper.

```
@GetMapping("/buscar/titular")
public ResponseEntity<TitularRecord> buscarTitular(
    @RequestParam TipoDocumento tipoDocumento,
    @RequestParam String documento
){
    try{
        TitularRecord titularRecord = service.buscarTitular(tipoDocumento, documento);
        return ResponseEntity.ok(titularRecord);
    } catch (IllegalArgumentException e){
        return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
        //Alternativa
        //return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
    }
}
```




```
@Component
public class TitularMapper {
    public TitularRecord toRecord(Titular titular) {
        return new TitularRecord(
            titular.getTipoDocumento(),
            titular.getDocumento(),
            titular.getNombre(),
            titular.getApellido(),
            new java.sql.Date(titular.getFechaNacimiento().getTime()),
            titular.getDireccion(),
            titular.getGrupoSanguineo(),
            titular.getFactorRH(),
            titular.getDonanteOrganos()
        );
    }

    public Titular toEntity(TitularRecord titularRecord) {
        Titular titular = new Titular();
        titular.setTipoDocumento(titularRecord.tipoDocumento());
        titular.setDocumento(titularRecord.documento());
        titular.setNombre(titularRecord.nombre());
        titular.setApellido(titularRecord.apellido());
        titular.setFechaNacimiento(titularRecord.fechaNacimiento());
        titular.setDireccion(titularRecord.direccion());
        titular.setGrupoSanguineo(titularRecord.grupoSanguineo());
        titular.setFactorRH(titularRecord.factorRH());
        titular.setDonanteOrganos(titularRecord.donanteOrganos());
        return titular;
    }
}
```

Comentario: Esta tarea no estaba prevista, llevó 2 hs.

ID HISTORIA: 2 - ID TAREA: 21

Se implementó la lógica que permite calcular automáticamente la vigencia de una licencia de conducir al momento de su emisión, basándose en la edad del titular y las reglas establecidas por la normativa vigente.

Se desarrolló el método `calcularVigenciaLicencia(Titular titular, String claseLicencia)` y se agregaron los cálculos correspondientes de la fecha de inicio y vencimiento en el método `emitirLicencia()`.

```
public int calcularVigenciaLicencia(Titular titular, String claseLicencia) {
    LocalDate fechaNacimientoTitular = titular.convertirDateALocalDate(titular.getFechaNacimiento());
    int edadTitular = Period.between(fechaNacimientoTitular, LocalDate.now()).getYears();

    int aniosVigencia = 0;

    if(edadTitular < 21) {
        boolean renovacion = false;

        for(Licencia l : titular.getLicencias()) {
            if(l.getClase().equalsIgnoreCase(claseLicencia)) {
                renovacion = true;
                break;
            }
        }

        if(renovacion) aniosVigencia = 3;
        else aniosVigencia = 1;
    }

    else if(edadTitular > 21 && edadTitular <= 46) aniosVigencia = 5;
    else if(edadTitular > 46 && edadTitular <= 60) aniosVigencia = 4;
    else if(edadTitular > 60 && edadTitular <= 70) aniosVigencia = 3;
    else if(edadTitular > 70) aniosVigencia = 1;

    return aniosVigencia;
}
```



Comentario: Llevó 1 hora, menos de lo estimado.

ID HISTORIA: 3 - ID TAREA: 22

Se implementó la lógica de negocio encargada de calcular el costo final de una licencia de conducir al momento de su emisión o renovación, conforme a las tarifas establecidas por el municipio.

Se desarrolló el método `calcularCostoLicencia(String clase, Integer vigencia)` en la clase `LicenciaServiceImpl` que retorna un `Float` correspondiente al costo de la licencia en base a la clase de la licencia y su vigencia, sumado a los costos fijos administrativos. Si la clase y/o la vigencia no es válida, lanza una excepción.

```
@Override
public Float calcularCostoLicencia(String clase, Integer vigencia) {
    // Validar valores nulos
    if (clase == null) {
        throw new IllegalArgumentException(s:"Clase de licencia no válida. Las clases válidas son: A, B, C, E, G");
    }
    if (vigencia == null) {
        throw new IllegalArgumentException(s:"Vigencia no válida. Las vigencias válidas son: 1, 3, 4, 5 años");
    }

    // Validar clase
    if (!CLASES_VALIDAS.contains(clase.toUpperCase())) {
        throw new IllegalArgumentException(s:"Clase de licencia no válida. Las clases válidas son: A, B, C, E, G");
    }

    // Validar vigencia
    if (!VIGENCIAS_VALIDAS.contains(vigencia)) {
        throw new IllegalArgumentException(s:"Vigencia no válida. Las vigencias válidas son: 1, 3, 4, 5 años");
    }

    Float costo = 8.00f;
```

```
        switch (clase.toUpperCase()) {
            case "A":
                if (vigencia == 5) costo += 40;
                else if (vigencia == 4) costo += 30;
                else if (vigencia == 3) costo += 25;
                else if (vigencia == 1) costo += 20;
                break;
            case "B":
                if (vigencia == 5) costo += 40;
                else if (vigencia == 4) costo += 30;
                else if (vigencia == 3) costo += 25;
                else if (vigencia == 1) costo += 20;
                break;
            case "C":
                if (vigencia == 5) costo += 47;
                else if (vigencia == 4) costo += 35;
                else if (vigencia == 3) costo += 30;
                else if (vigencia == 1) costo += 23;
                break;
            case "E":
                if (vigencia == 5) costo += 59;
                else if (vigencia == 4) costo += 44;
                else if (vigencia == 3) costo += 39;
                else if (vigencia == 1) costo += 29;
                break;
            case "G":
                if (vigencia == 5) costo += 40;
                else if (vigencia == 4) costo += 30;
                else if (vigencia == 3) costo += 25;
                else if (vigencia == 1) costo += 20;
                break;
        }

        return costo;
    }
}
```

Comentario: Empecé a las 12:00 hs y terminé a las 14:00 hs. Es decir, me llevó 2 hs aproximadamente. El task point estimado en un principio por mí (desarrollador) era de 3, es decir, 3 hs, pero me llevó un poco menos de tiempo terminarla, ya que con el uso de IA pude optimizar mi productividad.



ID HISTORIA: 3 - ID TAREA: 31

Se implementó un conjunto de pruebas unitarias para validar de forma integral y automática la lógica de cálculo del costo de una licencia de conducir, asegurando su correcto funcionamiento ante distintos escenarios de uso. Dichas pruebas estaban orientadas a verificar el cumplimiento de las reglas establecidas por la normativa vigente, que contempla la clase de licencia solicitada, la vigencia del permiso, y la aplicación de un cargo administrativo fijo. Además, se evaluó el comportamiento del sistema ante datos inválidos o inconsistentes.

Se creó una clase `LicenciaServiceImplTest` donde se desarrollaron todos los tests unitarios que aseguran el correcto funcionamiento de generar el costo de una licencia en base a la clase de la licencia y la vigencia de ésta.

Los tests desarrollados son los siguientes:

- `calcularCostoLicencia_casoValido_retornaCostoEsperado(String clase, int vigencia, float costoEsperado)`
- `calcularCostoLicencia_claseInvalida_lanzaExcepcion()`
- `calcularCostoLicencia_vigenciaInvalida_lanzaExcepcion()`
- `calcularCostoLicencia_claseMinuscula_funcionaCorrectamente()`
- `calcularCostoLicencia_claseNula_lanzaExcepcion()`
- `calcularCostoLicencia_vigenciaNula_lanzaExcepcion()`

```
@ParameterizedTest
@CsvSource({
    "A, 5, 48.0", // 8 + 40
    "A, 4, 38.0", // 8 + 30
    "A, 3, 33.0", // 8 + 25
    "A, 1, 28.0", // 8 + 20
    "B, 5, 48.0", // 8 + 40
    "B, 4, 38.0", // 8 + 30
    "B, 3, 33.0", // 8 + 25
    "B, 1, 28.0", // 8 + 20
    "C, 5, 55.0", // 8 + 47
    "C, 4, 43.0", // 8 + 35
    "C, 3, 38.0", // 8 + 30
    "C, 1, 31.0", // 8 + 23
    "E, 5, 67.0", // 8 + 59
    "E, 4, 52.0", // 8 + 44
    "E, 3, 47.0", // 8 + 39
    "E, 1, 37.0", // 8 + 29
    "G, 5, 48.0", // 8 + 40
    "G, 4, 38.0", // 8 + 30
    "G, 3, 33.0", // 8 + 25
    "G, 1, 28.0", // 8 + 20
})
void calcularCostoLicencia_casoValido_retornaCostoEsperado(String clase, int vigencia, float costoEsperado) {
    float costoCalculado = licenciaService.calcularCostoLicencia(clase, vigencia);
    assertEquals(costoEsperado, costoCalculado, 0.01,
        "El costo calculado para clase " + clase + " y vigencia " + vigencia + " años debe ser " + costoEsperado);
}

@Test
void calcularCostoLicencia_claseInvalida_lanzaExcepcion() {
    IllegalArgumentException exception = assertThrows(
        IllegalArgumentException.class,
        () -> licenciaService.calcularCostoLicencia(clase:"X", vigencia:5),
        "Debería lanzar excepción para clase inválida"
    );
    assertTrue(exception.getMessage().contains("Clase de licencia no válida"));
}
```



```
@Test
void calcularCostoLicencia_vigenciaInvalida_lanzaExcepcion() {
    IllegalArgumentException exception = assertThrows(
        IllegalArgumentException.class,
        () -> licenciaService.calcularCostoLicencia(clase:"A", vigencia:2),
        "Debería lanzar excepción para vigencia inválida"
    );

    assertTrue(exception.getMessage().contains("Vigencia no válida"));
}

@Test
void calcularCostoLicencia_claseMinuscula_funcionaCorrectamente() {
    float costoCalculado = licenciaService.calcularCostoLicencia(clase:"a", vigencia:5);
    assertEquals(48.0, costoCalculado, 0.01,
        "El costo debe ser calculado correctamente independientemente de mayúsculas/minúsculas");
}

@Test
void calcularCostoLicencia_claseNula_lanzaExcepcion() {
    IllegalArgumentException exception = assertThrows(
        IllegalArgumentException.class,
        () -> licenciaService.calcularCostoLicencia(clase:null, vigencia:5),
        "Debería lanzar excepción para clase nula"
    );

    assertTrue(exception.getMessage().contains("Clase de licencia no válida"));
}

@Test
void calcularCostoLicencia_vigenciaNula_lanzaExcepcion() {
    IllegalArgumentException exception = assertThrows(
        IllegalArgumentException.class,
        () -> licenciaService.calcularCostoLicencia(clase:"A", vigencia:null),
        "Debería lanzar excepción para vigencia nula"
    );

    assertTrue(exception.getMessage().contains("Vigencia no válida"));
}
```

Comentario: Empecé a las 14:00 hs y terminé a las 15:00 hs. Es decir, me llevó 1 h. aproximadamente. El task point estimado en un principio por mí (desarrollador) era de 2, es decir, 2 hs, pero me llevó un poco menos de tiempo terminarla, ya que con el uso de IA pude optimizar mi productividad.

ID HISTORIA: 4 - ID TAREA: 23

Se diseñaron las vistas de impresión de la licencia de conducir y del comprobante que serán entregadas al titular. Estas salidas incluyen todos los datos relevantes y están listas para ser enviadas a impresión directamente desde el sistema.




COMPROBANTE DE TRÁMITE: LICENCIA DE CONDUCIR
Dirección General de Tránsito y Transporte
Municipalidad de Santa Fe

Datos del titular

Nombre/s y apellido/s: Juan Pérez
Tipo de documento: DNI
Número de documento: 12345678

Datos de la licencia

Clase de licencia: B
Fecha de emisión: 2025-06-06
Fecha de vencimiento: 2030-12-31
Observaciones: Sin observaciones

Datos del trámite

Tipo de trámite: Emisión de licencia de conducir clase B
Costo del trámite: 48.0

Emitido en fecha: 2025-06-06

Firma y sello autorizado

[Imprimir comprobante](#)

LICENCIA DE CONDUCIR
Dirección General de Tránsito y Transporte
Municipalidad de Santa Fe

DATOS DEL TITULAR

NOMBRE/S Y APELLIDO/S: Juan Pérez
TIPO DE DOCUMENTO: DNI
NÚMERO DE DOCUMENTO: 12345678

DATOS DE LA LICENCIA

CLASE DE LICENCIA: B
FECHA DE EMISIÓN: 2025-06-06
FECHA DE VENCIMIENTO: 2030-12-31
OBSERVACIONES: Sin observaciones

Emitido en fecha: 2025-06-06

Firma y sello autorizado

[Imprimir licencia](#)

Comentario: Me llevó 1,5 hs. Menos tiempo del estimado.

ID HISTORIA: 9 - ID TAREA: 13

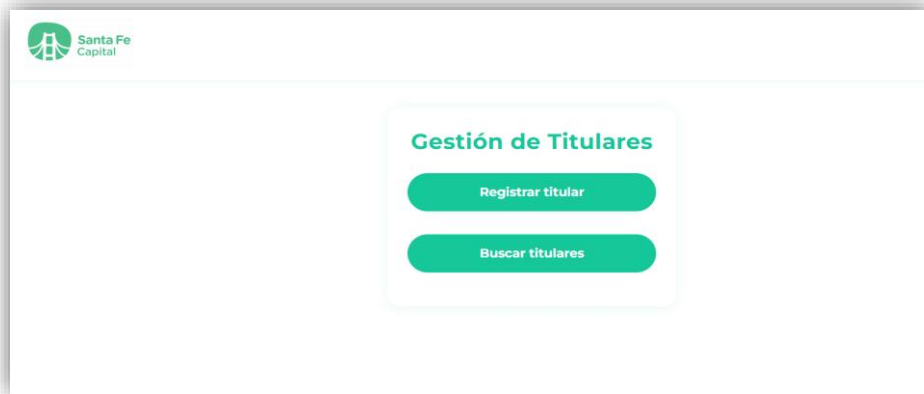
Se crearon las clases del modelo (Usuario, Titular, Trámite, Licencia) y el enum TipoDocumento con sus respectivos atributos y anotaciones.

Comentario: Llevó lo estimado.

ID HISTORIA: 9 - ID TAREA: 14

Se creó la vista menuTitular.html, la cual contiene las opciones “Registrar titular” y “Buscar titulares”. Se utilizó Thymeleaf y Bootstrap. Se conectó la interfaz con las demás interfaces.

```
<h2><b>Gestión de Titulares</b></h2>
<div class="d-grid gap-3">
  <a href="/api/titulares/registroTitular" class="btn btn-menu">Registrar titular</a>
  <a href="/api/titulares/buscarTitulares" class="btn btn-menu">Buscar titulares</a>
</div>
```



Comentario: Llevó una hora más de lo estimado. Se buscó asimilar los colores de la vista (y de todas las otras vistas) con los colores utilizados en la página oficial de obtención de licencias de la ciudad.

ID HISTORIA: 9 - ID TAREA: 15

Se desarrolló la interfaz donde se requieren los siguientes datos: tipo de documento (String), número de documento (int), apellido/s (String), nombre/s (String), fecha de nacimiento (Date), dirección (String), grupo sanguíneo (String), factor RH (String), donante de órganos (boolean).

Si el titular ya existe, hay campos vacíos o se ingresaron datos inválidos, se marcan los campos correspondientes y/o mostrar mensajes de error.

Para esta tarea también se implementó el TitularController que se encarga de mostrar la interfaz y comunicarse con ella.

```
@Tag(name = "Titular Controller", description = "Operaciones para la gestión de titulares")
@Controller
@RequestMapping("/api/titulares")
public class TitularController {

    private final TitularService service;
    private final TitularMapper titularMapper;

    @Autowired
    public TitularController(TitularService service, TitularMapper titularMapper) {
        this.service = service;
        this.titularMapper = titularMapper;
    }

    @GetMapping("/registroTitular")
    public String mostrarFormulario() { return "registroTitular"; }
}
```

Comentario: Llevó lo estimado.

ID HISTORIA: 9 - ID TAREA: 16

Se desarrolló la lógica en el backend necesaria para registrar un nuevo titular en el sistema, cumpliendo con los requisitos funcionales y de validación definidos en la historia de usuario.

Se implementaron las clases TitularRepository, interfaz TitularService, TitularServiceImpl:

La clase TitularRepository extiende de JpaRepository y define el siguiente método:

- existsByTipoDocumentoAndDocumento(TipoDocumento tipoDocumento, String documento)

La interfaz TitularService define los siguientes métodos:

- registrarTitular(Titular titular)
- listarTitulares()

La clase TitularServiceImpl implementa la interfaz TitularService.

En la clase TitularController se define el siguiente endpoint:

- registrarTitular(@RequestBody Titular titular)



```
@Operation(summary = "Registrar un titular",
            description = "Registra un nuevo titular en la base de datos",
            responses = {
                @ApiResponse(responseCode = "201", description = "Titular registrado correctamente"),
                @ApiResponse(responseCode = "400", description = "Error al registrar el titular"),
                @ApiResponse(responseCode = "500", description = "Error interno del servidor")
            })

@PostMapping("/registrar")
public ResponseEntity<Void> registrarTitular(@RequestBody @Valid TitularRecord titularRecord) {
    service.registrarTitular(titularRecord);
    return new ResponseEntity<>(HttpStatus.CREATED);
}

@Operation(
    summary = "Buscar un Titular",
    description = "Busca un titular por dtipo y numero de documento",
    responses = {
        @ApiResponse(responseCode = "202", description = "Titular no encontrado"),
        @ApiResponse(responseCode = "404", description = "Titular no encontrado"),
        @ApiResponse(responseCode = "400", description = "Parámetros Inválidos")
    }
)
```

```
@Override
@Transactional
public Titular registrarTitular(TitularRecord titularRecord) {

    if (titularRecord == null) {
        throw new IllegalArgumentException("El registro del titular no puede ser nulo");
    }

    // Verificar si ya existe un titular con el mismo documento
    if (repository.existsByTipoDocumentoAndDocumento(titularRecord.getTipoDocumento(), titularRecord.getDocumento())) {
        throw new IllegalArgumentException(
            "Ya existe un titular con documento: " + titularRecord.getDocumento() + " y tipo de documento: " + titularRecord.getTipoDocumento()
        );
    }

    // Convertir el Record a entidad
    Titular titular = titularMapper.toEntity(titularRecord);
    //sugerencia sacar por mapper: Titular titular = titularRecord.toTitular();
    // Guardar el titular en la base de datos
    return repository.save(titular);
}
```

Comentario: Empecé a las 14:00 hs y terminé a las 17:00 hs. Es decir, me llevó 3 hs aproximadamente. El task point estimado en un principio por mí (desarrollador) era de 2, es decir, 2 hs, pero me llevó un poco más de tiempo terminar la tarea. Esto se debió al proceso de aprendizaje requerido para las herramientas que empleamos, y a que no contaba con muchos conocimientos sobre el framework utilizado.

ID HISTORIA: 9 - ID TAREA: 17

Se implementó la suite de pruebas necesarias para verificar el correcto funcionamiento de la funcionalidad de alta de titulares, cubriendo las distintas capas de la aplicación (repository, service y controller) bajo la arquitectura definida. Las pruebas cubrieron tanto escenarios positivos como negativos.

Se generaron las clases TitularRepositoryTest, TitularServiceImplTest, TitularControllerTest (en el directorio “test” del proyecto) con los siguientes tests:

Tests de la clase TitularRepositoryTest:

- testExistsByTipoDocumentoAndDocumento(): verificar que el método existsByDocumento funciona correctamente.
- testSaveAndRetrieveTitular(): verificar que se puede guardar y recuperar un titular correctamente.
- testUniqueConstraintOnDocumento(): verificar que no se pueden guardar dos titulares con el mismo documento.



Tests de la clase TitularServiceImplTest:

- testCrearTitularExitoso()
- testCrearTitularDocumentoDuplicado()

Tests de la clase TitularControllerTest:

- testRegistrarTitular_Exitoso()
- testRegistrarTitular_DocumentoExistente()

```
@Test
void testExistsByTipoDocumentoAndDocumento() {
    // Verificar que el método existsByTipoDocumentoAndDocumento funciona correctamente
    boolean exists = titularRepository.existsByTipoDocumentoAndDocumento(TipoDocumento.DNI, documento:"12345678");
    assertTrue(exists, "El titular con DNI 12345678 debería existir en la base de datos");

    boolean notExists = titularRepository.existsByTipoDocumentoAndDocumento(TipoDocumento.DNI, documento:"87654321");
    assertFalse(notExists, "El titular con DNI 87654321 no debería existir en la base de datos");

    boolean notExistsDifferentType = titularRepository.existsByTipoDocumentoAndDocumento(TipoDocumento.PASAPORTE, documento:"12345678");
    assertFalse(notExistsDifferentType, "No debería existir un titular con pasaporte 12345678");
}

@Test
void testSaveAndRetrieveTitular() {
    // Verificar que se puede guardar y recuperar un titular correctamente
    Titular retrievedTitular = titularRepository.findById(titular.getIdTitular()).orElse(null);
    assertNotNull(retrievedTitular, "El titular debería haberse recuperado correctamente");
    assertEquals("Juan", retrievedTitular.getNombre(), "El nombre del titular debería ser 'Juan'");
    assertEquals("Perez", retrievedTitular.getApellido(), "El apellido del titular debería ser 'Perez'");
}

@Test
void testUniqueConstraintOnDocumento() {
    // Verificar que no se pueden guardar dos titulares con el mismo documento
    Titular otroTitular = new Titular();
    otroTitular.setTipoDocumento(TipoDocumento.DNI);
    otroTitular.setDocumento(documento:"12345678"); // Documento duplicado
    otroTitular.setNombre(nombre:"Maria");
    otroTitular.setApellido(apellido:"Gomez");
    otroTitular.setFechaNacimiento(new java.util.Date());
    otroTitular.setDireccion(direccion:"Calle Verdadera 456");
    otroTitular.setGrupoSanguineo(grupoSanguineo:"A+");
    otroTitular.setFactorRH(factorRH:"Negativo");
    otroTitular.setDonanteOrganos(donanteOrganos:false);

    Exception exception = assertThrows(Exception.class, () -> {
        titularRepository.save(otroTitular);
    });

    assertNotNull(exception, "Debería lanzarse una excepción al intentar guardar un documento duplicado");
}
```



```
@Test
void testCrearTitularExitoso() {
    // Arrange
    TitularRecord titularRecord = new TitularRecord(
        TipoDocumento.DNI,
        documento:"12345678",
        nombre:"Juan",
        apellido:"Pérez",
        Date.from(LocalDate.of(year:1990, month:1, dayOfMonth:1).atStartOfDay()).toInstant(java.time.ZoneOffset.UTC),
        direccion:"Calle Principal 123",
        grupoSanguineo:"A",
        factorRH:"Positivo",
        donanteOrganos:true
    );

    //Titular titularEsperado = titularRecord.toTitular();
    Titular titularEsperado = new Titular();
    titularEsperado.setTipoDocumento(TipoDocumento.DNI);
    titularEsperado.setDocumento(documento:"12345678");
    titularEsperado.setNombre(nombre:"Juan");
    titularEsperado.setApellido(apellido:"Pérez");
    titularEsperado.setFechaNacimiento(Date.from(LocalDate.of(year:1990, month:1, dayOfMonth:1).atStartOfDay()).toInstant(java.time.ZoneOffset.UTC));
    titularEsperado.setDireccion(direccion:"Calle Principal 123");
    titularEsperado.setGrupoSanguineo(grupoSanguineo:"A");
    titularEsperado.setFactorRH(factorRH:"Positivo");
    titularEsperado.setDonanteOrganos(donanteOrganos:true);

    when(titularMapper.toEntity(titularRecord)).thenReturn(titularEsperado);
    when(titularRepository.existsByTipoDocumentoAndDocumento(TipoDocumento.DNI, documento:"12345678")).thenReturn(false);
    when(titularRepository.save(any(Titular.class))).thenReturn(titularEsperado);

    // Act
    Titular titularCreado = titularService.registrarTitular(titularRecord);

    // Assert
    assertNotNull(titularCreado);
    assertEquals(titularRecord.tipoDocumento(), titularCreado.getTipoDocumento());
    assertEquals(titularRecord.documento(), titularCreado.getDocumento());
    assertEquals(titularRecord.nombre(), titularCreado.getNombre());
    assertEquals(titularRecord.apellido(), titularCreado.getApellido());
    assertEquals(titularRecord.fechaNacimiento(), titularCreado.getFechaNacimiento());
    assertEquals(titularRecord.direccion(), titularCreado.getDireccion());
    assertEquals(titularRecord.grupoSanguineo(), titularCreado.getGrupoSanguineo());
    assertEquals(titularRecord.factorRH(), titularCreado.getFactorRH());
    assertEquals(titularRecord.donanteOrganos(), titularCreado.getDonanteOrganos());

    // Verify
    verify(titularMapper).toEntity(titularRecord);
    verify(titularRepository).existsByTipoDocumentoAndDocumento(TipoDocumento.DNI, documento:"12345678");
    verify(titularRepository).save(titularEsperado);
}
```

```
@Test
void testCrearTitularDocumentoDuplicado() {
    // Arrange
    TitularRecord titularRecord = new TitularRecord(
        TipoDocumento.DNI,
        documento:"12345678",
        nombre:"Juan",
        apellido:"Pérez",
        Date.from(LocalDate.of(year:1990, month:1, dayOfMonth:1).atStartOfDay()).toInstant(java.time.ZoneOffset.UTC),
        direccion:"Calle Principal 123",
        grupoSanguineo:"A",
        factorRH:"Positivo",
        donanteOrganos:true
    );

    when(titularRepository.existsByTipoDocumentoAndDocumento(TipoDocumento.DNI, documento:"12345678")).thenReturn(true);

    // Act & Assert
    IllegalArgumentException exception = assertThrows(
        IllegalArgumentException.class,
        () -> titularService.registrarTitular(titularRecord),
        "Debería lanzar excepción cuando el documento ya existe"
    );

    assertTrue(exception.getMessage().contains(s:"Ya existe un titular con documento: 12345678 y tipo de documento: DNI"));

    // Verify
    verify(titularRepository).existsByTipoDocumentoAndDocumento(TipoDocumento.DNI, documento:"12345678");
    verify(titularRepository, never()).save(any(Titular.class));
}
```



```
@Test
void testRegistrarTitular_Exitoso() throws Exception {
    // Arrange
    when(titularService.registrarTitular(any(TitularRecord.class)))
        .thenReturn(titularEsperado);

    // Act & Assert
    mockMvc.perform(post("/api/titulares/registrar")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(titularValido)))
        .andExpect(status().isCreated());
}

@Test
void testRegistrarTitular_DocumentoExistente() throws Exception {
    // Arrange
    when(titularService.registrarTitular(any(TitularRecord.class)))
        .thenThrow(new IllegalArgumentException("Ya existe un titular con documento: " + titularValido.documento()));

    // Act & Assert
    mockMvc.perform(post("/api/titulares/registrar")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(titularValido)))
        .andExpect(status().isBadRequest());
}
```

Comentario: Empecé a las 17:00 hs y terminé a las 21:00 hs. Es decir, me llevó 4 hs aproximadamente. El task point estimado en un principio por mí (desarrollador) era de 2, es decir, 2 hs, pero me llevó un poco más de tiempo terminarla. Esto se debió al proceso de aprendizaje requerido para las herramientas que utilizamos, a que no contaba con muchos conocimientos del framework utilizado y nunca había hecho tests en otros proyectos. Además, se ocasionaron varios problemas y tuve que tomarme el tiempo de configurar cosas que desconocía con anterioridad.

ID HISTORIA: 9 - ID TAREA: 33

Se agregó la clase TitularRecord y se modificó el endpoint registrarTitular en TitularController. Se creó un manejador global de excepciones. Se corrigieron tests unitarios para adaptarlos a los cambios.

```
public record LicenciaRecord(
    @NotNull(message = "La clase de licencia es requerida")
    @Length(max = 1, message = "La clase de licencia debe tener como maximo 1 caracter")
    String clase,
    @Length(max = 100, message = "Las observaciones deben tener como maximo 100 caracteres")
    String observaciones,
    @NotNull(message = "El titular es requerido")
    TitularRecord titular
) {
    public Licencia toLicencia() {
        Licencia licencia = new Licencia();
        licencia.setClase(this.clase);
        licencia.setObservaciones(this.observaciones);
        licencia.setTitular(this.titular.toTitular());
        return licencia;
    }
}
```



```
@RestControllerAdvice
public class ControllerAdvisor {

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<ExceptionInfo> handleIllegalArgumentException(IllegalArgumentException ex, WebRequest request) {
        ExceptionInfo exceptionInfo = new ExceptionInfo(
            ex.getMessage(),
            request.getDescription(includeClientInfo:false),
            String.valueOf(System.currentTimeMillis()),
            HttpStatus.BAD_REQUEST.value()
        );
        return new ResponseEntity<>(exceptionInfo, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(EntityNotFoundException.class)
    public ResponseEntity<ExceptionInfo> handleEntityNotFoundException(EntityNotFoundException ex, WebRequest request) {
        ExceptionInfo exceptionInfo = new ExceptionInfo(
            ex.getMessage(),
            request.getDescription(includeClientInfo:false),
            String.valueOf(System.currentTimeMillis()),
            HttpStatus.NOT_FOUND.value()
        );
        return new ResponseEntity<>(exceptionInfo, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(Exception.class)
```

Comentario: Esta tarea no estaba prevista al comienzo del proyecto. Una vez comenzada, se estimó en 2 hs, pero finalmente tomó 4 hs, ya que, a medida que se realizaban cambios o se agregaban clases, surgían nuevos problemas.

DAILY SCRUM

Daily Meeting 1 - 08/05/2025.

Asistentes: equipo parcial (los disponibles en horario de la reunión).

Temas tratados:

- Avances en las tareas:
 - Codificación de clases del modelo y base de datos (ID Tarea 13).
 - Desarrollo de interfaz de menú de alta de titular (ID Tarea 14).
 - Desarrollo de interfaz de alta de titular con validaciones (ID Tarea 15).
- Dificultades:
 - Problemas en cuanto a la implementación de esta metodología ágil.
- Soluciones:
 - Colaboración entre miembros para resolver errores.
 - Consultas al Product Owner (profesor) para ajustes metodológicos.

Duración: 1 hora.

**Estado del sprint backlog:**

ID Historia	ID Tarea	Nombre tarea	Dueño	Esfuerzo	Estado
1	18	Diseñar interfaz de emisión de licencia	Aldana	2	En proceso
1	19	Desarrollar lógica de emisión de licencias y validación de datos	Lara y Aldana	5	En proceso
1	20	Realizar pruebas de emisión de licencias	Victoria	3	Pendiente
1	34	Desarrollar endpoint "buscarTitular"	Franco	2	Pendiente
2	21	Desarrollar lógica de cálculo de vigencia de una licencia	Aldana	2	Pendiente
3	22	Desarrollar lógica de cálculo de costo de una licencia	Santiago	3	Pendiente
3	31	Desarrollar test de la lógica de cálculo de costo de una licencia	Santiago	2	Pendiente
4	23	Diseñar las salidas a imprimir	Aldana	2	Pendiente
4	24	Desarrollar lógica de imprimir licencia	Franco	4	Pendiente
9	13	Codificar las clases del modelo y la base de datos	Franco	4	Finalizado
9	14	Diseñar interfaz de menú titulares	Franco	2	Finalizado
9	15	Diseñar interfaz de formulario de alta de titular con sus validaciones	Aldana	3	Finalizado
9	16	Desarrollar lógica de alta de titular y validación de datos	Santiago	2	En proceso
9	17	Realizar prueba de registro de titulares	Santiago	2	Pendiente
9	33	Refactorizar "registrarTitular" y "calcularCostoLicencia"	Santiago	2	Pendiente

*Tabla 3 – Estado del sprint backlog en daily meeting 1***Daily Meeting 2 - 15/05/2025.**

Asistentes: equipo completo.

Temas tratados:

- Progreso:
 - Desarrollo de lógica de alta de titular y validación de datos (ID Tarea 16).
 - Finalización de interfaz de emisión de licencias (ID Tarea 18).
 - Desarrollo de la lógica del cálculo de la vigencia de una licencia (ID Tarea 21).
 - Diseño de las salidas a imprimir finalizado (ID Tarea 23).
 - Desarrollo del endpoint buscarTitular (ID Tarea 34, no planificada inicialmente).
 -
- Desvíos:
 - Tarea con ID 19 (lógica de emisión de licencias) requirió más tiempo por complejidad en integración.
- Acuerdos:
 - Revisar gráfico Burndown para ajustar la velocidad.

Duración: 45 minutos.

**Estado del sprint backlog:**

ID Historia	ID Tarea	Nombre tarea	Dueño	Esfuerzo	Estado
1	18	Diseñar interfaz de emisión de licencia	Aldana	2	Finalizado
1	19	Desarrollar lógica de emisión de licencias y validación de datos	Lara y Aldana	5	En proceso
1	20	Realizar pruebas de emisión de licencias	Victoria	3	Pendiente
1	34	Desarrollar endpoint "buscarTitular"	Franco	2	Finalizado
2	21	Desarrollar lógica de cálculo de vigencia de una licencia	Aldana	2	Finalizado
3	22	Desarrollar lógica de cálculo de costo de una licencia	Santiago	3	En proceso
3	31	Desarrollar test de la lógica de cálculo de costo de una licencia	Santiago	2	Pendiente
4	23	Diseñar las salidas a imprimir	Aldana	2	Finalizado
4	24	Desarrollar lógica de imprimir licencia	Franco	4	Pendiente
9	13	Codificar las clases del modelo y la base de datos	Franco	4	Finalizado
9	14	Diseñar interfaz de menú titulares	Franco	2	Finalizado
9	15	Diseñar interfaz de formulario de alta de titular con sus validaciones	Aldana	3	Finalizado
9	16	Desarrollar lógica de alta de titular y validación de datos	Santiago	2	Finalizado
9	17	Realizar prueba de registro de titulares	Santiago	2	Pendiente
9	33	Refactorizar "registrarTitular" y "calcularCostoLicencia"	Santiago	2	Pendiente

*Tabla 4 – Estado del sprint backlog en daily meeting 2***Daily Meeting 3 - 22/05/2025.**

Asistentes: 3 de 5 integrantes.

Temas tratados:

- Estado actual:
 - Pruebas de registro de titulares (ID Tarea 17) atrasadas por problemas con JUnit.
 - Refactorización de código (ID Tarea 33) tomó 4 hs en lugar de 2 hs estimadas.
 - Desarrollo de la lógica del cálculo de costo de una licencia (ID Tarea 22).
- Bloqueadores:
 - Falta de conocimiento en testing.
- Acciones:
 - Ajustar estimaciones futuras considerando curva de aprendizaje.
- Acuerdos:
 - Finalizar pruebas unitarias de registro de titulares y de cálculo del costo de una licencia (ID Tarea 17, ID Tarea 31).

Duración: 30 minutos.

**Estado del sprint backlog:**

ID Historia	ID Tarea	Nombre tarea	Dueño	Esfuerzo	Estado
1	18	Diseñar interfaz de emisión de licencia	Aldana	2	Finalizado
1	19	Desarrollar lógica de emisión de licencias y validación de datos	Lara y Aldana	5	En proceso
1	20	Realizar pruebas de emisión de licencias	Victoria	3	Pendiente
1	34	Desarrollar endpoint "buscarTitular"	Franco	2	Finalizado
2	21	Desarrollar lógica de cálculo de vigencia de una licencia	Aldana	2	Finalizado
3	22	Desarrollar lógica de cálculo de costo de una licencia	Santiago	3	Finalizado
3	31	Desarrollar test de la lógica de cálculo de costo de una licencia	Santiago	2	En proceso
4	23	Diseñar las salidas a imprimir	Aldana	2	Finalizado
4	24	Desarrollar lógica de imprimir licencia	Franco	4	Pendiente
9	13	Codificar las clases del modelo y la base de datos	Franco	4	Finalizado
9	14	Diseñar interfaz de menú titulares	Franco	2	Finalizado
9	15	Diseñar interfaz de formulario de alta de titular con sus validaciones	Aldana	3	Finalizado
9	16	Desarrollar lógica de alta de titular y validación de datos	Santiago	2	Finalizado
9	17	Realizar prueba de registro de titulares	Santiago	2	En proceso
9	33	Refactorizar "registrarTitular" y "calcularCostoLicencia"	Santiago	2	Finalizado

*Tabla 5 – Estado del sprint backlog en daily meeting 3***Daily Meeting 4 - 29/05/2025.**

Asistentes: equipo completo.

Temas tratados:

- Cierre de Sprint 1:
 - 90% de tareas finalizadas (solo pendiente ajuste menor en impresión).
- Lecciones aprendidas:
 - Subestimación de tareas técnicas (ej.: tests).
- Preparación para Sprint 2:
 - Revisar historias de prioridad baja y dependencias.

Duración: 1 hora.



Estado del sprint backlog:

ID Historia	ID Tarea	Nombre tarea	Dueño	Esfuerzo	Estado
1	18	Diseñar interfaz de emisión de licencia	Aldana	2	Finalizado
1	19	Desarrollar lógica de emisión de licencias y validación de datos	Lara y Aldana	5	Finalizado
1	20	Realizar pruebas de emisión de licencias	Victoria	3	Finalizado
1	34	Desarrollar endpoint "buscarTitular"	Franco	2	Finalizado
2	21	Desarrollar lógica de cálculo de vigencia de una licencia	Aldana	2	Finalizado
3	22	Desarrollar lógica de cálculo de costo de una licencia	Santiago	3	Finalizado
3	31	Desarrollar test de la lógica de cálculo de costo de una licencia	Santiago	2	Finalizado
4	23	Diseñar las salidas a imprimir	Aldana	2	Finalizado
4	24	Desarrollar lógica de imprimir licencia	Franco	4	En proceso
9	13	Codificar las clases del modelo y la base de datos	Franco	4	Finalizado
9	14	Diseñar interfaz de menú titulares	Franco	2	Finalizado
9	15	Diseñar interfaz de formulario de alta de titular con sus validaciones	Aldana	3	Finalizado
9	16	Desarrollar lógica de alta de titular y validación de datos	Santiago	2	Finalizado
9	17	Realizar prueba de registro de titulares	Santiago	2	Finalizado
9	33	Refactorizar "registrarTitular" y "calcularCostoLicencia"	Santiago	2	Finalizado

Tabla 6 – Estado del sprint backlog en daily meeting 4

Nota sobre las daily meetings:

Si bien Scrum recomienda reuniones diarias, adaptamos la frecuencia a 1-2 reuniones semanales por restricciones de cursada. Solo documentamos las reuniones clave para el seguimiento, pero mantuvimos comunicación informal constante (grupo de WhatsApp) para resolver bloqueos rápidos. Esta flexibilidad permitió equilibrar la metodología con la disponibilidad del equipo.

REVISIÓN DEL SPRINT

Cada programador realizó la estimación del esfuerzo necesario para las tareas que le fueron asignadas. Esto se encuentra representado en la tabla, específicamente en la columna “**Estimado**”. Por otro lado, en la columna “**Real**” se visualiza el esfuerzo real demandado por cada tarea.

ID Historia	ID Tarea	Nombre tarea	Estimado	Real
1	18	Diseñar interfaz de emisión de licencia	2	2
1	19	Desarrollar lógica de emisión de licencias y validación de datos	5	7
1	20	Realizar pruebas de emisión de licencias	3	3
1	34	Desarrollar endpoint "buscarTitular"	2	2
2	21	Desarrollar lógica de cálculo de vigencia de una licencia	2	1
3	22	Desarrollar lógica de cálculo de costo de una licencia	3	2
3	31	Desarrollar test de la lógica de cálculo de costo de una licencia	2	1



4	23	Diseñar la salida impresa	2	2	
4	24	Desarrollar lógica de imprimir licencia	4	4	
9	13	Codificar las clases del modelo y la base de datos	4	4	
9	14	Diseñar interfaz de menú de titulares	2	3	
9	15	Diseñar interfaz de formulario de alta de titular con sus validaciones	3	3	
9	16	Desarrollar lógica de alta de titular y validación de datos	2	3	
9	17	Realizar prueba de registro de titulares	2	3	
9	33	Refactorizar "registrarTitular" y "calcularCostoLicencia"	2	4	
			40	45	TOTALES

Tabla 7 – Esfuerzo estimado por cada tarea vs. esfuerzo real

Viendo la tabla podemos concluir que este sprint requirió más esfuerzo del estimado, ya que, conforme avanzamos en el desarrollo de las historias, notamos la necesidad de incluir algunas tareas más aparte de las previstas. Además, fue necesario investigar el modo de uso de las herramientas, adquirir más conocimientos acerca del framework, entre otros contratiempos propios de cada tarea que ya fueron detallados en la sección “Ejecución - Código fuente por tarea”.

GRÁFICO BURNDOWN

Debido al desconocimiento que teníamos sobre Jira, no completamos las estimaciones de las historias de usuario en el sistema antes de iniciar el sprint. Esto provocó que el gráfico burndown no se vea correctamente.

Por lo tanto, para este sprint decidimos realizar dicho gráfico utilizando Excel:

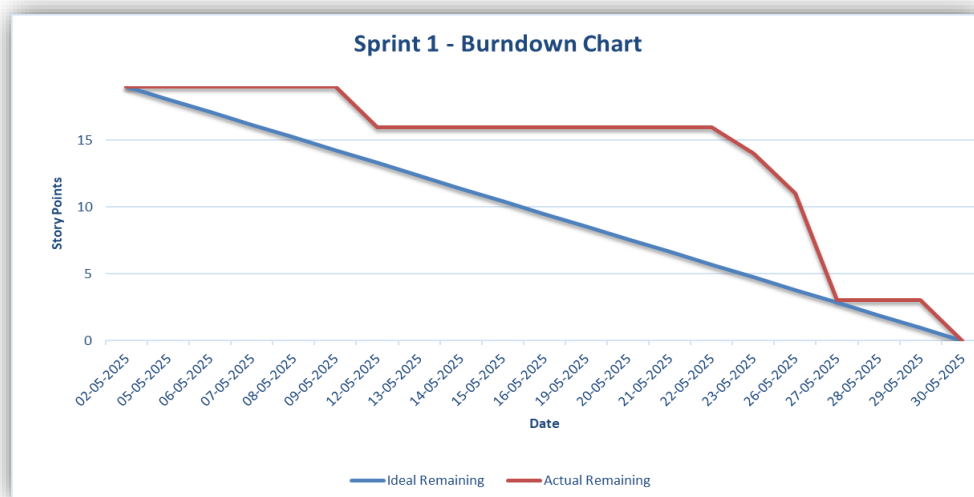


Gráfico burndown del sprint 1



En el gráfico se puede observar que logramos completar todas las historias de usuario comprometidas dentro del plazo establecido, aunque durante la primera mitad del sprint no se reflejaron avances significativos en la cantidad de puntos. Esto se debió a que muchas tareas estaban en progreso o finalizadas pero aún no permitían cerrar historias completas. En la última semana, la curva muestra un descenso marcado, que refleja el esfuerzo coordinado del equipo para cerrar todas las historias pendientes a tiempo.

El comportamiento de la curva no fue cercano a la línea ideal, pero sí logramos alcanzar el objetivo final. Esto indica que, aunque hubo momentos de pausa o demoras en la integración de tareas, pudimos organizarnos y enfocar el trabajo en la finalización efectiva hacia el cierre del sprint.

Consideramos que las estimaciones realizadas fueron razonables, ya que ninguna historia quedó fuera del alcance ni se generaron desvíos importantes en la carga total de trabajo. Para el próximo sprint, el desafío es lograr una distribución más pareja en la entrega de historias, sin acumular tantos cierres en los últimos días.

En resumen, fue un sprint positivo: se cumplió con todo lo planificado y se demostró capacidad de respuesta y organización para alcanzar las metas, incluso en los momentos de mayor carga.

PLANIFICACIÓN - SPRINT 2

La fecha de inicio del desarrollo del sprint 2 es: 09/06/2025.

DIVISIÓN DE TAREAS

A continuación, se presenta una tabla con las tareas necesarias para cada historia, junto con el programador asignado y la cantidad de task points necesarios para su realización. Los posibles estados son: “**Pendiente**”, “**En proceso**” y “**Finalizado**”.

ID Historia	ID Tarea	Nombre tarea	Dueño	Esfuerzo	Estado
8	37	Diseñar interfaz para la búsqueda de licencias vigentes.	Aldana	1	Finalizado
8	38	Implementar lógica de búsqueda de licencias vigentes por criterios.	Aldana	2	Finalizado
8	39	Realizar pruebas de funcionalidad de búsqueda de licencias.	Aldana	2	Finalizado
8	62	Diseñar interfaz de menú de licencias.	Aldana	1	Finalizado
8	66	Refactorizar "buscarLicenciasVigentes".	Aldana	1	Finalizado
7	40	Desarrollar lógica para listar licencias expiradas.	Santiago	4	Finalizado
7	41	Diseñar interfaz de listado de licencias expiradas.	Franco	2	Finalizado



7	42	Realizar pruebas de listado de licencias expiradas.	Santiago	1	Finalizado
5	25	Diseñar interfaz de renovación de licencia.	Aldana	1	Finalizado
5	26	Desarrollar lógica de renovación de licencias.	Aldana	2	Finalizado
5	27	Realizar pruebas de renovación de licencias.	Victoria	2	Finalizado
5	67	Refactorizar "renovarLicencia".	Victoria	1	Finalizado
6	43	Desarrollar la funcionalidad para solicitar copia de una licencia.	Aldana	2	Finalizado
6	45	Realizar pruebas de emisión de copias de licencias.	Franco	1	Finalizado
6	68	Refactorizar "emitirCopiaLicencia".	Franco	1	Finalizado
10	52	Diseñar interfaces para modificar un titular con sus validaciones.	Victoria	1	Finalizado
10	54	Desarrollar lógica de modificación de datos de un titular.	Aldana	1	Finalizado
10	63	Desarrollar lógica para buscar titulares.	Franco	2	Finalizado
10	58	Realizar pruebas de búsqueda y modificación de titulares.	Lara	2	Finalizado
11	48	Implementar lógica de registro de un usuario administrativo.	Franco	2	Finalizado
11	50	Diseñar interfaces para el registro de un usuario administrativo con sus validaciones.	Victoria	1	Finalizado
11	49	Realizar pruebas de creación de usuarios administrativos.	Lara	1	Finalizado
13	46	Diseñar interfaz para login de usuarios (administradores y administrativos).	Aldana	1	Finalizado
13	47	Desarrollar lógica para login de usuarios.	Franco	2	Finalizado
13	59	Realizar pruebas de seguridad.	Lara	2	Finalizado
13	61	Diseñar interfaces de menú para usuarios administradores y administrativos.	Aldana	1	Finalizado
12	55	Desarrollar lógica para modificar un usuario administrativo.	Santiago	2	Finalizado
12	56	Diseñar interfaces para modificar datos de un usuario administrativo con sus validaciones.	Aldana	1	Finalizado
12	65	Desarrollar lógica para buscar usuarios administrativos.	Santiago	2	Finalizado
12	57	Realizar pruebas de búsqueda y modificación de usuarios administrativos.	Santiago	1	Finalizado

Tabla 8 – División de tareas del sprint 2

EJECUCIÓN

En esta sección se describe con detalle cómo se implementó cada tarea, y se incluyen algunos comentarios de los desarrolladores que justifican los motivos de ciertas desviaciones respecto a las estimaciones realizadas.



ID HISTORIA: 5 - ID TAREA: 25

Se desarrolló la interfaz donde se observan en modo lectura los datos correspondientes a la licencia que se quiere renovar y el motivo de dicha renovación (cambio de datos del titular o vigencia de licencia vencida). Se conectó dicha interfaz con el LicenciaController, donde además se desarrolló el método validarRenovacion(TipoDocumento tipoDocumento, String documento, String claseLicencia) que verifica, antes de mostrar la interfaz, si la licencia seleccionada puede ser renovada o no (para ello se comprueba que los datos del titular asociado hayan sido modificados, por lo que fue necesario agregar a la clase Titular un atributo booleano que almacena dicha información).

```
@GetMapping("/validarRenovacion")
public ResponseEntity<> validarRenovacion(
    @RequestParam TipoDocumento tipoDocumento,
    @RequestParam String documento,
    @RequestParam String claseLicencia
) {
    Titular titular = titularService.buscarTitularDocumento(tipoDocumento, documento);
    Licencia licencia = licenciaService.buscarLicenciaPorTitularyClase(titularService.buscarTitularDocumento(tipoDocumento, documento), claseLicencia);

    if (!licenciaService.sePuedeRenovar(licencia, titular)) {
        return ResponseEntity.badRequest().body(Map.of(
            "message", "La licencia no está vencida ni hubo modificaciones en los datos del titular asociado. No se puede renovar."
        ));
    }

    return ResponseEntity.ok().build();
}

@GetMapping("/renovacionLicencia")
public String mostrarFormularioRenovacion(
    @RequestParam TipoDocumento tipoDocumento,
    @RequestParam String documento,
    @RequestParam String claseLicencia,
    @RequestParam String motivo,
    Model model
) {
    Titular titular = titularService.buscarTitularDocumento(tipoDocumento, documento);
    Licencia licencia = licenciaService.buscarLicenciaPorTitularyClase(titular, claseLicencia);
    if (licencia == null) {
        throw new RuntimeException(HttpStatus.NOT_FOUND, reason: "Licencia no encontrada");
    }

    model.addAttribute(attributeName: "licencia", licencia);
    model.addAttribute(attributeName: "titular", licencia.getTitular());
    model.addAttribute(attributeName: "motivoRenovacion", motivo);

    return "renovacionLicencia";
}
```

```
@GetMapping("/comprobanteCopia")
public String mostrarComprobanteCopia(
    @RequestParam("idTramite") Integer idTramite,
    Model model) {

    Tramite tramite = tramiteService.buscarPorId(idTramite);

    model.addAttribute(attributeName: "tramite", tramite);
    model.addAttribute(attributeName: "titular", tramite.getTitularAsociado());
    model.addAttribute(attributeName: "licencia", tramite.getLicenciaAsociada());

    return "comprobanteTramite";
}
```



The screenshot shows a web form titled "Renovación de Licencia" from Santa Fe Capital. The form contains the following fields:

- Nombre y apellido del titular:** Marcos Perez
- Tipo de documento:** DNI
- Número de documento:** 99999999
- Clase de licencia:** A
- Observaciones:** (Empty text area)
- Motivo de renovación:** Cambio de datos del titular

At the bottom of the form is a green button labeled "Confirmar renovación".

The screenshot shows a receipt titled "COMPROBANTE DE TRÁMITE: LICENCIA DE CONDUCIR" from Santa Fe Capital. The receipt is issued by the "Dirección General de Tránsito y Transporte" of the "Municipalidad de Santa Fe". It contains the following information:

Datos del titular	
Nombre/s y apellido/s:	Marcos Perez
Tipo de documento:	DNI
Número de documento:	99999999

Datos de la licencia	
Clase de licencia:	A
Fecha de emisión:	2025-06-21
Fecha de vencimiento:	2030-02-15
Observaciones:	Sin observaciones

Datos del trámite	
Tipo de trámite:	Renovación de licencia de conducir clase A
Costo del trámite:	48.0

Emitido en fecha: 2025-06-21

Firma y sello autorizado

At the bottom is a green button labeled "Imprimir comprobante".

ID HISTORIA: 5 - ID TAREA: 26

Se desarrolló la lógica que permite renovar una licencia. Se agregó el método `renovarLicencia(TipoDocumento tipoDocumento, String documento, String claseLicencia)` al `LicenciaController`, donde se busca la licencia a renovar y se pasa al método `renovarLicencia(Licencia licencia)` del `LicenciaService` para que actualice la fecha de vigencia de la licencia en la base de datos y genere el trámite correspondiente.



```
@PostMapping("/renovar")
public ResponseEntity<?> renovarLicencia(
    @RequestParam TipoDocumento tipoDocumento,
    @RequestParam String documento,
    @RequestParam String claseLicencia
) {
    Titular titular = titularService.buscarTitularDocumento(tipoDocumento, documento);
    Licencia licencia = licenciaService.buscarLicenciaPorTitularyClase(titular, claseLicencia);

    if (licencia == null) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(Map.of(k1:"message", v1:"Licencia no encontrada"));
    }

    try {
        Tramite tramite = licenciaService.renovarLicencia(licencia);
        titularService.actualizarModificaciones(licencia.getTitular());
        return ResponseEntity.ok(Map.of(
            k1:"message", v1:"Licencia renovada correctamente",
            k2:"idTramite", tramite.getIdTramite()
        ));
    } catch (Exception e) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(Map.of(k1:"message", v1:"Error al renovar la licencia"));
    }
}
```

```
@Override
public Tramite renovarLicencia(Licencia licencia) {
    // Calcular años de vigencia según titular y clase de licencia
    int añosVigencia = calcularVigenciaLicencia(licencia.getTitular(), licencia.getClase());

    int añoVencimiento = LocalDate.now().getYear() + añosVigencia;

    // Convertir fecha de nacimiento (Date) a LocalDate
    LocalDate fechaNacimiento = ((java.sql.Date) licencia.getTitular().getFechaNacimiento()).toLocalDate();

    // Crear la fecha de vencimiento con mismo mes y día que el nacimiento, pero en el año calculado
    LocalDate fechaVencimiento = LocalDate.of(
        añoVencimiento,
        fechaNacimiento.getMonth(),
        fechaNacimiento.getDayOfMonth()
    );

    // Actualizar la fecha de vencimiento
    licencia.setFechaVencimiento(java.sql.Date.valueOf(fechaVencimiento));

    // Guardar la licencia actualizada
    repository.save(licencia);

    //Registro del trámite
    Tramite tramite = new Tramite();
    tramite.setFecha(java.sql.Date.valueOf(java.time.LocalDate.now()));
    tramite.setDescripcion("Renovación de licencia de conducir clase " + licencia.getClase());
    tramite.setCosto(calcularCostoLicencia(licencia.getClase(), añosVigencia));
    tramite.setTitularAsociado(licencia.getTitular());
    tramite.setUsuarioResponsable(usuarioService.buscarUsuarioPorId(idUsuario:1)); // POR AHORA
    tramite.setLicenciaAsociada(licencia);
    return tramiteService.registrarTramite(tramite);
}
```

ID HISTORIA: 5 - ID TAREA: 27

Se desarrollaron tests para probar la renovación de licencias.

En la clase LicenciaServiceImplTest se realizaron los siguientes tests:

- renovarLicencia_actualizaVigenciaYRegistraTramite()
- sePuedeRenovar_licenciaVigente_titularNoModificado_retornaFalse()

En la clase LicenciaControllerTest se realizó el siguiente test:

- validarRenovacion_licenciaVigente_sinCambiosEnTitular_retornaBadRequest()
- validarRenovacion_titularModificado_licenciaVigente_retornaOk()



```
@Test
void renovarLicencia_actualizaVigenciaYRegistraTramite() {
    // Setup de titular y licencia vencida
    Titular titular = new Titular();
    titular.setNombre(nombre:"Laura");
    titular.setApellido(apellido:"Sosa");
    titular.setFechaNacimiento(Date.valueOf(LocalDate.of(year:1990, month:6, dayOfMonth:15)));

    Licencia licencia = new Licencia();
    licencia.setClase(clase:"B");
    licencia.setTitular(titular);
    licencia.setFechaVencimiento(Date.valueOf(LocalDate.of(year:2020, month:6, dayOfMonth:15)));

    // Trámite esperado
    Tramite tramiteMock = new Tramite();
    tramiteMock.setIdTramite(idTramite:123);
    tramiteMock.setDescripcion(descripcion:"Renovación de licencia de conducir clase B");

    // Mock comportamiento de usuario y trámite
    when(usuarioService.buscarUsuarioPorId(idUsuario:1)).thenReturn(value:null); // no importa usuario por ahora
    when(tramiteService.registrarTramite(any(type:Tramite.class))).thenReturn(tramiteMock);

    // Ejecutar renovación
    Tramite resultado = licenciaService.renovarLicencia(licencia);

    // Verificaciones
    assertNotNull(resultado);
    assertEquals(expected:"Renovación de licencia de conducir clase B", resultado.getDescripcion());

    // Validar que la licencia se haya actualizado con una nueva fecha
    LocalDate vencimientoEsperado = LocalDate.of(
        LocalDate.now().getYear() + 5,
        month:6, dayOfMonth:15
    );
    assertEquals(Date.valueOf(vencimientoEsperado), licencia.getFechaVencimiento());
}

@Test
void sePuedeRenovar_licenciaVigente_titularNoModificado_retornaFalse() {
    // Arrange
    Titular titular = new Titular();
    titular.setModificado(modificado:false); // No hubo cambios en los datos del titular

    Licencia licencia = new Licencia();
    licencia.setEstaVigente(estaVigente:true); // La licencia sigue vigente
    licencia.setTitular(titular);

    // Act
    boolean resultado = licenciaService.sePuedeRenovar(licencia, titular);

    // Assert
    assertFalse(resultado, message:"No debe poder renovarse si la licencia está vigente y el titular no fue modificado");
}
```



```
@Test
void validarRenovacion_licenciaVigente_sinCambiosEnTitular_retornaBadRequest() throws Exception {
    TipoDocumento tipoDocumento = TipoDocumento.DNI;
    String documento = "12345678";
    String claseLicencia = "B";

    Titular titular = new Titular();
    titular.setTipoDocumento(tipoDocumento);
    titular.setDocumento(documento);
    titular.setModificado(modificado:false); // No fue modificado

    Licencia licencia = new Licencia();
    licencia.setClase(claseLicencia);
    licencia.setTitular(titular);
    licencia.setEstaVigente(estaVigente:true); // Aún está vigente

    when(titularService.buscarTitularDocumento(tipoDocumento, documento)).thenReturn(titular);
    when(licenciaService.buscarLicenciaPorTitularClase(titular, claseLicencia)).thenReturn(licencia);
    when(licenciaService.sePuedeRenovar(licencia, titular)).thenReturn(value:false);

    mockMvc.perform(get(uriTemplate:"/api/licencias/validarRenovacion")
        .param(name:"tipoDocumento", tipoDocumento.name())
        .param(name:"documento", documento)
        .param(name:"claseLicencia", claseLicencia))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath(expression:"$.message").value(expectedValue:"La licencia no está vencida ni hubo modificaciones en los datos del titular asociado. No se puede renovar."));
}

@Test
void validarRenovacion_titularModificado_licenciaVigente_retornaOk() throws Exception {
    TipoDocumento tipoDocumento = TipoDocumento.DNI;
    String documento = "12345678";
    String claseLicencia = "B";

    Titular titular = new Titular();
    titular.setTipoDocumento(tipoDocumento);
    titular.setDocumento(documento);
    titular.setModificado(modificado:true); // Fue modificado

    Licencia licencia = new Licencia();
    licencia.setClase(claseLicencia);
    licencia.setTitular(titular);
    licencia.setEstaVigente(estaVigente:true); // Sigue vigente, pero se permite renovar

    when(titularService.buscarTitularDocumento(tipoDocumento, documento)).thenReturn(titular);
    when(licenciaService.buscarLicenciaPorTitularClase(titular, claseLicencia)).thenReturn(licencia);
    when(licenciaService.sePuedeRenovar(licencia, titular)).thenReturn(value:true);

    mockMvc.perform(get(uriTemplate:"/api/licencias/validarRenovacion")
        .param(name:"tipoDocumento", tipoDocumento.name())
        .param(name:"documento", documento)
        .param(name:"claseLicencia", claseLicencia))
        .andExpect(status().isOk());
}
```

ID HISTORIA: 5 - ID TAREA: 67

Se refactorizó el método renovarLicencia(), Se simplificó dicho método a través de dos métodos privados reutilizables:

- Para la creación de los trámites: crearTramite(String descripción, Float costo, Titular titular, Licencia licencia).
- Para el cálculo de la fecha de vencimiento: calcularFechaVencimiento(Titular titular, int añosVigencia).

De esta forma se evita la duplicación de código. También se simplificó el método sePuedeRenovar().

```
private Tramite crearTramite(String descripcion, Float costo, Titular titular, Licencia licencia) {
    Tramite tramite = new Tramite();
    tramite.setFecha(java.sql.Date.valueOf(java.time.LocalDate.now()));
    tramite.setDescripcion(descripcion);
    tramite.setCosto(costo);
    tramite.setTitularAsociado(titular);
    tramite.setUsuarioResponsable(usuarioService.buscarUsuarioPorId(idUsuario:1)); // POR AHORA
    tramite.setLicenciaAsociada(licencia);
    return tramite;
}
```




```

public Boolean sePuedeRenovar(Licencia licencia, Titular titular) {
    return licencia != null && (!licencia.getEstaVigente() || titular.getModificado());
}

@Override
public Tramite renovarLicencia(Licencia licencia) {
    int aniosVigencia = calcularVigenciaLicencia(licencia.getTitular(), licencia.getClase());
    licencia.setFechaVencimiento(java.sql.Date.valueOf(calcularFechaVencimiento(licencia.getTitular(), aniosVigencia)));
    repository.save(licencia);

    String descripcion = "Renovación de licencia de conducir clase " + licencia.getClase();
    Tramite tramite = crearTramite(descripcion, calcularCostoLicencia(licencia.getClase(), aniosVigencia), licencia.getTitular(), licencia);
    return tramiteService.registrarTramite(tramite);
}

private LocalDate calcularFechaVencimiento(Titular titular, int aniosVigencia) {
    LocalDate fechaNacimiento = ((java.sql.Date) titular.getFechaNacimiento()).toLocalDate();
    int anioVencimiento = LocalDate.now().getYear() + aniosVigencia;
    return LocalDate.of(anioVencimiento, fechaNacimiento.getMonth(), fechaNacimiento.getDayOfMonth());
}

```

ID HISTORIA: 6 - ID TAREA: 43

Se desarrolló la funcionalidad para generar la copia de una licencia. Dicha funcionalidad permite imprimir la licencia (se muestra la salida diseñada en la historia N°4) y genera en la base de datos el trámite con el nro. de copia emitida.

```

@GetMapping("/imprimirCopiaLicencia")
public String mostrarCopiaLicencia(
    @RequestParam("tipoDocumento") TipoDocumento tipoDocumento,
    @RequestParam("documento") String documento,
    @RequestParam("claseLicencia") String claseLicencia,
    Model model) {

    // Buscar titular
    Titular titular = titularService.buscarTitularDocumento(tipoDocumento, documento);
    Licencia licencia = licenciaService.buscarLicenciaPorTitularyClase(titular, claseLicencia);

    model.addAttribute(attributeName:"titular", titular);
    model.addAttribute(attributeName:"licencia", licencia);

    return "impresionLicencia";
}

```

LICENCIA DE CONDUCIR	
Dirección General de Tránsito y Transporte Municipalidad de Santa Fe	
DATOS DEL TITULAR	
NOMBRE/S Y APELLIDO/S:	Franco Ocampo
TIPO DE DOCUMENTO:	DNI
NÚMERO DE DOCUMENTO:	22333333
DATOS DE LA LICENCIA	
CLASE DE LICENCIA:	A
FECHA DE EMISIÓN:	2025-05-31
FECHA DE VENCIMIENTO:	2030-07-09
OBSERVACIONES:	Sin observaciones
Emitido en fecha: 2025-05-31	Firma y sello autorizado

Imprimir licencia





COMPROBANTE DE TRÁMITE: LICENCIA DE CONDUCIR
 Dirección General de Tránsito y Transporte
 Municipalidad de Santa Fe

Datos del titular	
Nombre/s y apellido/s:	Franco Ocampo
Tipo de documento:	DNI
Número de documento:	22333333
Datos de la licencia	
Clase de licencia:	A
Fecha de emisión:	2025-05-31
Fecha de vencimiento:	2030-07-09
Observaciones:	Sin observaciones
Datos del trámite	
Tipo de trámite:	Emisión de copia de licencia clase A: duplicado
Costo del trámite:	50.0

Emitido en fecha: 2025-05-31

Firma y sello autorizado

Imprimir comprobante

ID HISTORIA: 6 - ID TAREA: 45

Se realizaron tests para la emisión de copias de licencias. Se creó la clase `LicenciaControllerTest` para probar también el controlador.

Tests desarrollados:

- `emitirCopiaLicencia()` en `LicenciaControllerTest`.
- `emitirCopiaLicencia_registraTramiteCorrectamente()` en `LicenciaServiceImplTest`.

```
@Test
void emitirCopiaLicencia() throws Exception {
    TipoDocumento tipoDocumento = TipoDocumento.DNI;
    String documento = "12345678";
    String claseLicencia = "B";

    Titular titular = new Titular();
    titular.setTipoDocumento(tipoDocumento);
    titular.setDocumento(documento);

    Licencia licencia = new Licencia();
    licencia.setClase(claseLicencia);
    licencia.setTitular(titular);

    Tramite tramite = new Tramite();
    tramite.setIdTramite(idTramite:1);

    when(titularService.buscarTitularDocumento(tipoDocumento, documento)).thenReturn(titular);
    when(licenciaService.buscarLicenciaPorTitularYClase(titular, claseLicencia)).thenReturn(licencia);
    when(licenciaService.emitirCopiaLicencia(licencia, titular)).thenReturn(tramite);

    mockMvc.perform(get(UriTemplate:"/api/licencias/emitirCopia")
        .param(name:"tipoDocumento", tipoDocumento.name())
        .param(name:"documento", documento)
        .param(name:"claseLicencia", claseLicencia)
        .accept(MediaType.APPLICATION_JSON))
        .andExpect(status().isOk())
        .andExpect(jsonPath(expression:"$.idTramite").value(expectedValue:1))
        .andExpect(jsonPath(expression:"$.tipoDocumento").value(tipoDocumento.name()))
        .andExpect(jsonPath(expression:"$.documento").value(documento))
        .andExpect(jsonPath(expression:"$.claseLicencia").value(claseLicencia));
}
```



```
@Test
void emitirCopiaLicencia_registraTramiteCorrectamente() {
    Titular titular = new Titular();
    titular.setNombre(nombre:"Juan");
    titular.setApellido(apellido:"Pérez");

    Licencia licencia = new Licencia();
    licencia.setClase(clase:"B");
    licencia.setTitular(titular);

    Tramite tramiteMock = new Tramite();
    tramiteMock.setIdTramite(idTramite:1);
    tramiteMock.setDescripcion(descripcion:"Emisión de copia de licencia clase B: duplicado");

    when(tramiteService.contarCopiasPorTitularYClase(titular, claseLicencia:"B")).thenReturn(value:0);
    when(usuarioService.buscarUsuarioPorId(idUsuario:1)).thenReturn(value:null); // Simulamos que devuelve null
    when(tramiteService.registrarTramite(any(type:Tramite.class))).thenReturn(tramiteMock);

    Tramite resultado = licenciaService.emitirCopiaLicencia(licencia, titular);

    assertNotNull(resultado);
    assertEquals(expected:"Emisión de copia de licencia clase B: duplicado", resultado.getDescripcion());
    assertEquals(expected:1, resultado.getIdTramite());
}
```

ID HISTORIA: 6 - ID TAREA: 68

Se refactorizó el método emitirCopiaLicencia(), utilizando el método privado desarrollado para la creación de trámites (crearTramite()).

```
@Override
public Tramite emitirCopiaLicencia(Licencia licencia, Titular titular) {
    int cantidadCopias = tramiteService.contarCopiasPorTitularYClase(titular, licencia.getClase());
    String descripcion = switch (cantidadCopias) {
        case 0 -> "Emisión de copia de licencia clase " + licencia.getClase() + ": duplicado";
        case 1 -> "Emisión de copia de licencia clase " + licencia.getClase() + ": triplicado";
        case 2 -> "Emisión de copia de licencia clase " + licencia.getClase() + ": cuadruplicado";
        default -> "Emisión de copia de licencia clase " + licencia.getClase() + ": " + (cantidadCopias + 2) + "e copia";
    };
    Tramite tramite = crearTramite(descripcion, costo:50f, titular, licencia);
    return tramiteService.registrarTramite(tramite);
}
```

ID HISTORIA: 7 - ID TAREA: 40

Se desarrolló la lógica de negocio en la capa de servicio que permite recuperar un listado de licencias de conducir cuyo estado actual sea “no vigente”, es decir, que se encuentran expiradas a la fecha de consulta. El método resultante retorna una estructura de datos que contiene, los siguientes campos para cada licencia vencida:

- Nombre y apellido del titular.
- Tipo de documento del titular.
- Número de documento del titular.
- Clase de licencia.
- Fecha de vencimiento.
- Estado actual.



```
@GetMapping("/noVigentes")
public ResponseEntity<List<LicencialistadoRecord>> mostrarLicenciasNoVigentes(
    @RequestParam(required = false) @DateTimeFormat(pattern = "yyyy-MM-dd") Date fechaDesde,
    @RequestParam(required = false) @DateTimeFormat(pattern = "yyyy-MM-dd") Date fechaHasta,
    @RequestParam(required = false) String clase) {

    List<LicencialistadoRecord> licencias = licenciaService.buscarLicenciasNoVigentes(fechaDesde, fechaHasta, clase);
    return ResponseEntity.ok(licencias);
}
```

```
@Override
public List<LicencialistadoRecord> buscarLicenciasNoVigentes(Date fechaDesde, Date fechaHasta, String clase) {
    Date hoy = java.sql.Date.valueOf(java.time.LocalDate.now());

    List<LicencialistadoRecord> licenciasRecord = new ArrayList<>();

    List<Licencia> licencias = repository.findLicenciasNoVigentes(hoy, clase);
    /*
     * SOLUCIÓN PROVISORIA: Tuve inconvenientes con la consulta de JPA
     * que no filtraba correctamente por fecha de vencimiento.
     * Habría que revisar la consulta para que funcione correctamente.
     * Por ahora, filtramos por fechas en memoria (esto es muy ineficiente
     * si hay muchas licencias, pero es una solución provisoria).
     */
    List<Licencia> filtradas = buscarLicenciasPorFechas(fechaDesde, fechaHasta, licencias);

    for(Licencia l : filtradas) {
        LicencialistadoRecord lr = new LicencialistadoRecord(
            l.getTitular().getNombre() + " " + l.getTitular().getApellido(),
            l.getTitular().getTipoDocumento(),
            l.getTitular().getDocumento(),
            l.getClase(),
            estadoActual:"No vigente",
            l.getFechaVencimiento()
        );
        licenciasRecord.add(lr);
    }

    return licenciasRecord;
}
```

```
@Query("SELECT l FROM Licencia l WHERE l.fechaVencimiento < :hoy " +
    "AND (:clase IS NULL OR l.clase = :clase)")
List<Licencia> findLicenciasNoVigentes(
    @Param("hoy") Date hoy,
    @Param("clase") String clase
);
```

Comentario: Esta tarea me llevó todo un día porque surgieron problemas con el filtrado de las licencias vencidas en la base de datos. Lo que me generaba problemas fueron los formatos de las fechas definidos. Luego de una jornada de intentar llegar a la solución, encontré una solución (un poco ineficiente) que permite al usuario realizar lo solicitado. Por lo tanto, no se cumplió con los tasks points estimados previamente.

ID HISTORIA: 7 - ID TAREA: 41

Se desarrolló la interfaz para listar licencias no vigentes. En la misma se muestran el nombre y apellido del titular, documento del titular, clase de licencia, estado y fecha de vencimiento de la misma. Se



permite filtrar por rango de fechas y/o clase de licencia. Se conectó dicha interfaz con el LicenciaController.

```
@GetMapping("/listarLicenciasNoVigentes")
public String mostrarBusquedaLicenciasNoVigentes() {
    return "licenciasNoVigentes";
}
```

Buscar licencias vencidas

Fecha desde: 20/01/2020 Fecha hasta: 04/07/2025

Clase de licencia: Todos, Clase A, **Clase B**, Clase C

Buscar

Nombre y apellido	Documento	Clase	Estado	Vencimiento
Santiago Nicle	42558687	B	No vigente	31/01/2023
Lucas Gauna	87654321	B	No vigente	30/04/2023

Renovar

ID HISTORIA: 7 - ID TAREA: 42

Se desarrollaron pruebas unitarias para validar la correcta ejecución de la lógica encargada de recuperar licencias de conducir vencidas, garantizando que se respeten los criterios de filtrado definidos (rango de fechas, clase de licencia) y la exclusión de licencias en proceso de renovación. Las pruebas deben asegurar que el comportamiento del sistema sea consistente y confiable frente a distintos escenarios.

```
@Test
void mostrarLicenciasNoVigentes_devuelveListaFiltrada() throws Exception {
    // Arrange
    LicencialistadoRecord l1 = new LicencialistadoRecord(nombreCompletoTitular:"Juan Pérez", TipoDocumento.DNI, documento:"12345678", clase:"A", estad:"No vigente", java.sql.Date.valueOf("2024-01-10"));
    LicencialistadoRecord l2 = new LicencialistadoRecord(nombreCompletoTitular:"Ana Gómez", TipoDocumento.DNI, documento:"33239454", clase:"B", estad:"No vigente", java.sql.Date.valueOf("2024-03-15"));
    List<LicencialistadoRecord> mockList = List.of(l1, l2);

    when(licenciaService.buscarLicenciasNoVigentes(fechaDesde:null, fechaHasta:null, clase:null)).thenReturn(mockList);

    // Act & Assert
    mockMvc.perform(get(uriTemplate:"/api/licencias/noVigentes")
        .accept(MediaType.APPLICATION_JSON))
        .andExpect(status().isOk())
        .andExpect(jsonPath(expression:"${0}.nombreCompletoTitular").value(expectedValue:"Juan Pérez"))
        .andExpect(jsonPath(expression:"${0}.clase").value(expectedValue:"A"))
        .andExpect(jsonPath(expression:"${1}.nombreCompletoTitular").value(expectedValue:"Ana Gómez"))
        .andExpect(jsonPath(expression:"${1}.clase").value(expectedValue:"B"));
}
```



```

1  @Test
2  void testFindLicenciasNoVigentes() {
3      Date hoy = new Date();
4      Calendar cal = Calendar.getInstance();
5
6      // Licencia vencida (no vigente)
7      Licencia licenciaVencida = new Licencia();
8      licenciaVencida.setTitular(titular);
9      licenciaVencida.setClase("A");
10     cal.setTime(hoy);
11     cal.add(Calendar.YEAR, -2);
12     licenciaVencida.setFechaInicio(cal.getTime());
13     cal.add(Calendar.YEAR, 1);
14     licenciaVencida.setFechaVencimiento(cal.getTime()); // vencida hace 1 año
15     licenciaVencida.setEstaVigente(false);
16     licenciaVencida.setObservaciones("Vencida");
17     licenciaRepository.save(licenciaVencida);
18
19     // Licencia vencida de otra clase
20     Licencia licenciaVencidaB = new Licencia();
21     licenciaVencidaB.setTitular(titular);
22     licenciaVencidaB.setClase("B");
23     cal.setTime(hoy);
24     cal.add(Calendar.YEAR, -3);
25     licenciaVencidaB.setFechaInicio(cal.getTime());
26     cal.add(Calendar.YEAR, 1);
27     licenciaVencidaB.setFechaVencimiento(cal.getTime()); // vencida hace 2 años
28     licenciaVencidaB.setEstaVigente(false);
29     licenciaVencidaB.setObservaciones("Vencida B");
30     licenciaRepository.save(licenciaVencidaB);
31
32     licenciaRepository.flush();
33
34     // Test: debe devolver ambas licencias vencidas
35     List<Licencia> noVigentes = licenciaRepository.findLicenciasNoVigentes(hoy, null);
36     assertTrue(noVigentes.stream().anyMatch(l -> l.getClase().equals("A")));
37     assertTrue(noVigentes.stream().anyMatch(l -> l.getClase().equals("B")));
38
39     // Test: filtrar por clase B
40     List<Licencia> noVigentesB = licenciaRepository.findLicenciasNoVigentes(hoy, "B");
41     assertEquals(1, noVigentesB.size());
42     assertEquals("B", noVigentesB.get(0).getClase());
43 }

```

```

@Test
void buscarLicenciasNoVigentes_filtrarPorFechasYClase() {
    // Arrange
    Titular titular = new Titular();
    titular.setNombre(nombre:"Juan");
    titular.setApellido(apellido:"Pérez");
    Licencia l1 = new Licencia();
    l1.setTitular(titular);
    l1.setIdLicencia(idLicencia:1);
    l1.setClase(clase:"A");
    l1.setFechaVencimiento(java.sql.Date.valueOf(s:"2024-01-10"));
    Licencia l2 = new Licencia();
    l2.setTitular(titular);
    l2.setIdLicencia(idLicencia:2);
    l2.setClase(clase:"B");
    l2.setFechaVencimiento(java.sql.Date.valueOf(s:"2024-03-15"));
    Licencia l3 = new Licencia();
    l3.setTitular(titular);
    l3.setIdLicencia(idLicencia:3);
    l3.setClase(clase:"A");
    l3.setFechaVencimiento(java.sql.Date.valueOf(s:"2025-01-10")); // fuera de rango

    // Solo l1 y l2 deben ser devueltas por el mock
    when(licenciaRepository.findLicenciasNoVigentes(any(type:Date.class), any())).thenReturn(List.of(l1, l2, l3));

    Date desde = java.sql.Date.valueOf(s:"2024-01-01");
    Date hasta = java.sql.Date.valueOf(s:"2024-12-31");

    // Act
    List<LicenciaListadoRecord> resultado = licenciaService.buscarLicenciasNoVigentes(desde, hasta, clase:null);

    // Assert
    assertEquals(expected:2, resultado.size());
    assertEquals(expected:"No vigente", resultado.get(index:0).estadoActual());
    assertEquals(expected:"Juan Pérez", resultado.get(index:0).nombreCompletoTitular());

    // Filtrar por clase
    List<LicenciaListadoRecord> soloClaseA = licenciaService.buscarLicenciasNoVigentes(desde, hasta, clase:"A");
    assertEquals(expected:2, soloClaseA.size()); // El mock devuelve l1 y l3, pero solo l1 está en rango
    assertTrue(resultado.stream().anyMatch(r -> r.nombreCompletoTitular().equals(anObject:"Juan Pérez")));
}

```



Comentario: Empecé a las 12:30 hs y terminé a las 13:00 hs. Es decir, me llevó 30 minutos aproximadamente, por lo que se sobrecumplió el objetivo.

ID HISTORIA: 8 - ID TAREA: 37

Se desarrolló la interfaz donde se pueden ingresar los siguientes datos para la búsqueda de licencias vigentes: nombre y apellido del titular (búsqueda parcial o exacta), grupo sanguíneo y factor RH, donación de órganos (sí/no). El listado incluye datos del titular (nombre y apellido, documento, fecha de nacimiento, dirección), clase de licencia y fecha de vencimiento, e información relevante asociada al criterio de búsqueda. Además, es posible seleccionar una licencia para renovarla o emitir una copia de la misma.

Se conectó la interfaz con el LicenciaController.

```
@GetMapping("/listarLicenciasVigentes")
public String mostrarBusquedaLicencias() {
    return "licenciasVigentes";
}
```

ID HISTORIA: 8 - ID TAREA: 38

Se llevó a cabo la lógica que permite buscar licencias vigentes por nombre y/o apellido del titular, grupo sanguíneo, factor RH y donación de órganos.

Se agregó el método `findByFechaVencimientoAfter(LocalDate fecha)` en el `LicenciaRepository`, que devuelve todas las licencias cuya fecha de vencimiento es posterior a la fecha en que se realiza la búsqueda.

Se agregó también el método `buscarLicenciasVigentes(String nombreApellido, String grupoSanguineo, String factorRH, boolean donanteOrganos)` en el `LicenciaService`, que devuelve la lista de licencias que cumplen con los criterios especificados. Si no hay criterios, se devuelven todas las licencias vigentes existentes.



```
@Override
public List<LicenciaRecord> buscarLicenciasVigentes(String nombreApellido, String grupoSanguineo, String factorRH, boolean donanteOrganos)
{
    LocalDate hoy = LocalDate.now();
    List<Licencia> vigentes = repository.findByFechaVencimientoAfter(hoy);

    return vigentes.stream()
        .filter(licencia -> {
            Titular t = licencia.getTitular();
            boolean coincideNombre = (nombreApellido == null || nombreApellido.isBlank()) ||
                t.getNombre().toLowerCase().contains(nombreApellido.toLowerCase()) ||
                t.getApellido().toLowerCase().contains(nombreApellido.toLowerCase());
            boolean coincideGrupo = (grupoSanguineo == null || grupoSanguineo.isBlank()) ||
                grupoSanguineo.equalsIgnoreCase(t.getGrupoSanguineo());
            boolean coincideRH = (factorRH == null || factorRH.isBlank()) ||
                factorRH.equalsIgnoreCase(t.getFactorRH());
            boolean coincideDonante = !donanteOrganos || t.getDonanteOrganos();
            return coincideNombre && coincideGrupo && coincideRH && coincideDonante;
        })
        .map(licencia -> {
            Titular t = licencia.getTitular();
            TitularRecord titularRecord = new TitularRecord(
                t.getTipoDocumento(),
                t.getDocumento(),
                t.getNombre(),
                t.getApellido(),
                t.getFechaNacimiento(),
                t.getDireccion(),
                t.getGrupoSanguineo(),
                t.getFactorRH(),
                t.getDonanteOrganos()
            );
            return new LicenciaRecord(
                licencia.getClase(),
                licencia.getObservaciones(),
                titularRecord,
                new java.sql.Date(licencia.getFechaVencimiento().getTime())
            );
        })
        .toList();
}
```



Buscar licencias vigentes

Nombre y/o apellido

Grupo sanguíneo

Factor RH

☐ Sólo donantes de órganos

Nombre y apellido	Documento	Fecha de nacimiento	Dirección	Grupo Sanguíneo	RH	Donante	Clase de licencia	Vencimiento
Juan Massin	12345633	07/03/2000	Calle Falsa 123	A	+	Sí	A	07/03/2030
Juan Massin	12345633	07/03/2000	Calle Falsa 123	A	+	Sí	B	07/03/2030



Santa Fe Capital

Buscar licencias vigentes

Nombre y/o apellido:

Grupo sanguíneo:

Factor RH:

☐ Sólo donantes de órganos

Nombre y apellido	Documento	Fecha de nacimiento	Dirección	Grupo Sanguíneo	RH	Donante	Clase de licencia	Vencimiento
Juan Massin	12345633	07/03/2000	Calle Falsa 123	A	+	Sí	A	07/03/2030
Juan Massin	12345633	07/03/2000	Calle Falsa 123	A	+	Sí	B	07/03/2030

Santa Fe Capital

localhost:8080 dice

No se encontraron licencias con esos criterios.

Buscar licencias vigentes

Nombre y/o apellido:

Grupo sanguíneo:

Factor RH:

☐ Sólo donantes de órganos

ID HISTORIA: 8 - ID TAREA: 39

Se agregaron a la clase `LicenciaServiceImplTest` pruebas de búsqueda por nombre y donación de órganos, pruebas de búsqueda sin coincidencias y pruebas de búsqueda por grupo sanguíneo.



```
@Test
void buscarLicenciasVigentes_filtrarPorNombreYDonante() {
    Titular t = new Titular();
    t.setTipoDocumento(TipoDocumento.DNI);
    t.setDocumento(documento:"12345678");
    t.setNombre(nombre:"Ana");
    t.setApellido(apellido:"Martínez");
    t.setFechaNacimiento(Date.valueOf(LocalDate.of(year:1990, month:1, dayOfMonth:1)));
    t.setDireccion(direccion:"Calle A");
    t.setGrupoSanguineo(grupoSanguineo:"A");
    t.setFactorRH(factorRH:"+");
    t.setDonanteOrganos(donanteOrganos:true);

    Licencia l = new Licencia();
    l.setClase(clase:"B");
    l.setObservaciones(observaciones:"Obs");
    l.setFechaVencimiento(Date.valueOf(LocalDate.now().plusDays(daysToAdd:90)));
    l.setTitular(t);

    when(licenciaRepository.findByFechaVencimientoAfter(any(LocalDate.class)))
        .thenReturn(List.of(l));

    List<LicenciaRecord> resultado = licenciaService
        .buscarLicenciasVigentes(nombreApellido:"ana", grupoSanguineo:"A", factorRH:"+", donante:true);

    assertEquals(1, resultado.size());
    assertEquals("Ana", resultado.get(index:0).titular().nombre());
}

@Test
void buscarLicenciasVigentes_sinCoincidencias_retornaListaVacía() {
    when(licenciaRepository.findByFechaVencimientoAfter(any(LocalDate.class)))
        .thenReturn(List.of());

    List<LicenciaRecord> resultado = licenciaService
        .buscarLicenciasVigentes(nombreApellido:"Pedro", grupoSanguineo:"0", factorRH:"0", donante:false);

    assertTrue(resultado.isEmpty());
}
```

```
@Test
void buscarLicenciasVigentes_filtrarSoloGrupoSanguineo() {
    Titular a = new Titular();
    a.setNombre(nombre:"Mario"); a.setGrupoSanguineo(grupoSanguineo:"0"); a.setFactorRH(factorRH:"+");
    a.setFechaNacimiento(Date.valueOf(LocalDate.of(year:1980, month:5, dayOfMonth:1))); a.setDonanteOrganos(donanteOrganos:false);
    Licencia la = new Licencia(); la.setClase(clase:"A");
    la.setFechaVencimiento(Date.valueOf(LocalDate.now().plusMonths(monthsToAdd:6))); la.setTitular(a);

    Titular b = new Titular();
    b.setNombre(nombre:"Silvia"); b.setGrupoSanguineo(grupoSanguineo:"A"); b.setFactorRH(factorRH:"+");
    b.setFechaNacimiento(Date.valueOf(LocalDate.of(year:1990, month:7, dayOfMonth:1))); b.setDonanteOrganos(donanteOrganos:false);
    Licencia lb = new Licencia(); lb.setClase(clase:"B");
    lb.setFechaVencimiento(Date.valueOf(LocalDate.now().plusMonths(monthsToAdd:6))); lb.setTitular(b);

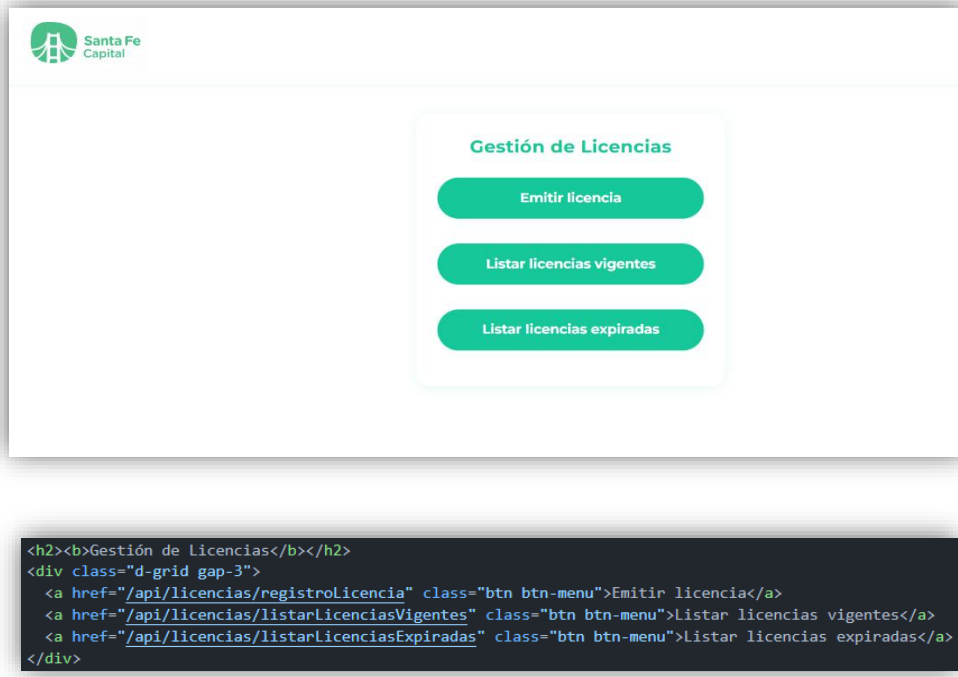
    when(licenciaRepository.findByFechaVencimientoAfter(any(LocalDate.class)))
        .thenReturn(List.of(la, lb));

    List<LicenciaRecord> r = licenciaService.buscarLicenciasVigentes(
        nombreApellido:"", // nombreApellido
        grupoSanguineo:"0", // grupoSanguineo
        factorRH:"", // factorRH
        donanteOrganos:false); // donanteOrganos

    assertEquals(1, r.size());
    assertEquals("Mario", r.get(index:0).titular().nombre());
}
```

ID HISTORIA: 8 - ID TAREA: 62

Se creó la vista menuLicencia.html, la cual contiene las opciones “Emitir licencia”, “Listar licencias vigentes” y “Listar licencias expiradas”. Se utilizó Thymeleaf y Bootstrap. Se conectó la interfaz con las demás interfaces.

ID HISTORIA: 8 - ID TAREA: 66

Se refactorizó el método buscarLicenciasVigentes(), haciendo el filtrado más preciso y permitiendo que la modificación o el agregado de filtros sea más sencillo.



```
@Override
public List<LicenciaRecord> buscarLicenciasVigentes(String nombreApellido, String grupoSanguineo, String factorRH, boolean donanteOrganos) {
    LocalDate hoy = LocalDate.now();
    List<Licencia> vigentes = repository.findByFechaVencimientoAfter(hoy);

    return vigentes.stream()
        .filter(licencia -> filtraPorNombreApellido(licencia, nombreApellido))
        .filter(licencia -> filtraPorGrupoSanguineo(licencia, grupoSanguineo))
        .filter(licencia -> filtraPorFactorRH(licencia, factorRH))
        .filter(licencia -> !donanteOrganos || Boolean.TRUE.equals(licencia.getTitular().getDonanteOrganos()))
        .map(licencia -> {
            Titular t = licencia.getTitular();
            TitularRecord titularRecord = new TitularRecord(
                t.getTipoDocumento(),
                t.getDocumento(),
                t.getNombre(),
                t.getApellido(),
                t.getFechaNacimiento(),
                t.getDireccion(),
                t.getGrupoSanguineo(),
                t.getFactorRH(),
                t.getDonanteOrganos()
            );
            return new LicenciaRecord(
                licencia.getClase(),
                licencia.getObservaciones(),
                titularRecord,
                new java.sql.Date(licencia.getFechaVencimiento().getTime())
            );
        })
        .toList();
}

private boolean filtraPorNombreApellido(Licencia licencia, String nombreApellido) {
    if (nombreApellido == null || nombreApellido.isBlank()) return true;
    Titular t = licencia.getTitular();
    String nombreCompleto = (t.getNombre() + " " + t.getApellido()).toLowerCase();
    return nombreCompleto.contains(nombreApellido.toLowerCase());
}

private boolean filtraPorGrupoSanguineo(Licencia licencia, String grupoSanguineo) {
    if (grupoSanguineo == null || grupoSanguineo.isBlank()) return true;
    return grupoSanguineo.equalsIgnoreCase(licencia.getTitular().getGrupoSanguineo());
}

private boolean filtraPorFactorRH(Licencia licencia, String factorRH) {
    if (factorRH == null || factorRH.isBlank()) return true;
    return factorRH.equalsIgnoreCase(licencia.getTitular().getFactorRH());
}
```

ID HISTORIA: 10 - ID TAREA: 52

Se desarrolló la interfaz para modificar los datos de un titular existente. En la misma se encuentran bloqueados los campos para el tipo y número de documento. Se solicitan los mismos datos que para el registro, y se valida que todos los campos obligatorios estén completos y sean válidos.

Además, se desarrolló la interfaz para buscar titulares. Allí se observan todos los datos de cada titular y se puede seleccionar uno para modificarlo. También se puede filtrar por apellido, tipo de documento y/o número de documento del titular.

Se conectaron las interfaces con el TitularController, donde se agregaron los métodos mostrarFormularioListar() y mostrarFormularioModificar(TipoDocumento tipoDocumento, String documento, Model model).

```
//listar
@GetMapping("/buscarTitulares")
public String mostrarFormularioListar() { return "listaTitulares"; }
```



```
@GetMapping("/modificar/{tipoDocumento}/{documento}")
public String mostrarFormularioModificar(
    @PathVariable TipoDocumento tipoDocumento,
    @PathVariable String documento,
    Model model) {

    try {
        Titular titular = service.buscarTitularDocumento(tipoDocumento, documento);
        if (titular == null) {
            throw new EntityNotFoundException(message:"Titular no encontrado");
        }
        model.addAttribute(attributeName:"titular", titular);
        return "modificacionTitular";

    } catch (EntityNotFoundException e) {
        return "error/404";
    } catch (Exception e) {
        return "error/500";
    }
}
```

 Santa Fe Capital

Buscar titulares

Apellido: Tipo de documento: Número de documento:

Tipo Doc	Documento	Nombre	Apellido	Fecha nacimiento	Dirección	Grupo	RH	Donante
DNI	12345633	Juan	Massin	03/03/2000	Calle Falsa 123	A	+	Sí

 Santa Fe Capital

localhost:8080 dice
No se encontraron titulares con esos criterios.

Buscar titulares

Apellido: Tipo de documento: Número de documento:



 Santa Fe Capital

Modificar titular

Tipo de documento *	Número de documento *
DNI	12345633
Apellido/s *	Nombre/s *
Massin	Juan
Fecha de nacimiento *	Dirección *
07/03/2000	Calle Falsa 123
Grupo sanguíneo *	Factor RH *
A	Positivo (+)

Donante de órganos ☒

[Modificar titular](#)

 Santa Fe Capital

localhost:8080 dice

Por favor, complete todos los campos obligatorios correctamente y use sólo letras en nombre y apellido.

[Aceptar](#)

Modificar titular

Tipo de documento *	Número de documento
DNI	12345633
Apellido/s *	Nombre/s *
Massin44	
Solo se permiten letras y espacios.	Este campo es obligatorio.
Fecha de nacimiento	Dirección
05/03/2000	San Martín 1530
Grupo sanguíneo	Factor RH
A	Positivo (+)

Donante de órganos ☐

[Modificar titular](#)



localhost:8080 dice
Titular modificado correctamente.

Modificar titular

Tipo de documento * Número de documento

Apellido/s Nombre/s

Fecha de nacimiento Dirección

Grupo sanguíneo Factor RH

Donante de órganos ☐

Modificar titular

ID HISTORIA: 10 - ID TAREA: 63

Se desarrolló la lógica para buscar titulares por apellido, tipo de documento y/o número de documento. En el TitularService se agregó el método buscarTitulares(String apellido, TipoDocumento tipoDocumento, String documento) y en el TitularController se agregó el método listarOBuscarTitulares(String apellido, TipoDocumento tipoDocumento, String documento).

```
@Override
public List<Titular> buscarTitulares(String apellido, TipoDocumento tipoDocumento, String documento)
    List<Titular> todos = repository.findAll();

    return todos.stream()
        .filter(t -> {
            boolean coincideApellido = (apellido == null || apellido.isBlank()) ||
                t.getApellido().toLowerCase().contains(apellido.toLowerCase());
            boolean coincideTipoDoc = (tipoDocumento == null) ||
                tipoDocumento.equals(t.getTipoDocumento());
            boolean coincideDocumento = (documento == null || documento.isBlank()) ||
                documento.equals(t.getDocumento());
            return coincideApellido && coincideTipoDoc && coincideDocumento;
        })
        .toList();
}
```



```

@GetMapping(value = "/lista", produces = "application/json")
@ResponseBody
@Operation(summary = "Listar / Buscar Titulares",
description = "Lista todos los titulares o filtra por apellido, tipoDocumento y/o documento",
responses = {
    @ApiResponse(responseCode = "200", description = "Lista obtenida"),
    @ApiResponse(responseCode = "500", description = "Error interno")
})
public ResponseEntity<List<TitularRecord>> listarOBuscarTitulares(
    @RequestParam(required = false) String apellido,
    @RequestParam(required = false) TipoDocumento tipoDocumento,
    @RequestParam(required = false) String documento) {

    List<TitularRecord> resultado = service.buscarTitulares(apellido, tipoDocumento, documento)
        .stream()
        .map(titularMapper::toRecord)
        .toList();

    return ResponseEntity.ok(resultado);
}

```

ID HISTORIA: 10 - ID TAREA: 54

Se desarrolló la lógica para modificar un titular y guardar los cambios en la base de datos. Se agregó el método actualizarTitular(TipoDocumento tipoDocumento, String documento, TitularRecord titularModificado) en el TitularService y se agregó el método modificarTitular(TipoDocumento tipoDocumento, String documento, TitularRecord titularModificado) en el TitularController. No se podrá modificar el tipo de documento ni el número de documento del titular.

```

@PutMapping("/modificar/{tipoDocumento}/{documento}")
public ResponseEntity<Void> modificarTitular(
    @PathVariable TipoDocumento tipoDocumento,
    @PathVariable String documento,
    @RequestBody @Valid TitularRecord titularModificado) {

    try {
        service.actualizarTitular(tipoDocumento, documento, titularModificado);
        return ResponseEntity.noContent().build();
    } catch (EntityNotFoundException e) {
        return ResponseEntity.notFound().build();
    } catch (Exception e) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
    }
}

```

```

public void actualizarTitular(TipoDocumento tipoDocumento, String documento, TitularRecord titularModificado) {
    Titular titularExistente = repository
        .findByTipoDocumentoAndDocumento(tipoDocumento, documento)
        .orElseThrow(() -> new EntityNotFoundException(message: "Titular no encontrado"));

    // Actualizar campos (excepto tipo y nro documento)
    titularExistente.setNombre(titularModificado.nombre());
    titularExistente.setApellido(titularModificado.apellido());
    titularExistente.setFechaNacimiento(titularModificado.fechaNacimiento());
    titularExistente.setDireccion(titularModificado.direccion());
    titularExistente.setGrupoSanguineo(titularModificado.grupoSanguineo());
    titularExistente.setFactorRH(titularModificado.factorRH());
    titularExistente.setDonanteOrganos(titularModificado.donanteOrganos());

    repository.save(titularExistente);
}

```


ID HISTORIA: 10 - ID TAREA: 58

Se realizaron las pruebas necesarias para la búsqueda y modificación de titulares. Dado que son muchas, se adjuntan algunas.

En TitularControllerTest:

```
@Test
void listarTitulares_conFiltrosCombinados_retornaTitularesFiltrados() throws Exception {
    // Arrange
    String apellido = "Pérez";
    TipoDocumento tipoDocumento = TipoDocumento.DNI;
    String documento = "12345678";

    Titular titular = new Titular();
    titular.setTipoDocumento(tipoDocumento.DNI);
    titular.setDocumento(documento:"12345678");
    titular.setNombre(nombre:"Juan");
    titular.setApellido(apellido:"Pérez");

    TitularRecord titularRecord = new TitularRecord(
        tipoDocumento.DNI, documento:"12345678", nombre:"Juan", apellido:"Pérez",
        java.sql.Date.valueOf(s:"1990-01-01"), direccion:"Calle 1", grupoSanguíneo:"A", factorRH:"+", donante:true
    );

    when(titularService.buscarTitulares(apellido, tipoDocumento, documento)).thenReturn(List.of(titular));
    when(titularMapper.toRecord(titular)).thenReturn(titularRecord);

    // Act & Assert
    mockMvc.perform(get(uriTemplate:"/api/titulares/lista")
        .param(name:"apellido", apellido)
        .param(name:"tipoDocumento", tipoDocumento.name())
        .param(name:"documento", documento)
        .accept(MediaType.APPLICATION_JSON))
        .andExpect(status().isOk())
        .andExpect(jsonPath(expression:"$", hasSize(1)))
        .andExpect(jsonPath(expression:"$[0].apellido").value(expectedValue:"Pérez"))
        .andExpect(jsonPath(expression:"$[0].tipoDocumento").value(expectedValue:"DNI"))
        .andExpect(jsonPath(expression:"$[0].documento").value(expectedValue:"12345678"));
}

@Test
void listarTitulares_sinResultados_retornaListaVacía() throws Exception {
    // Arrange
    String apellido = "Inexistente";

    when(titularService.buscarTitulares(apellido, tipoDocumento:null, documento:null)).thenReturn(List.of());

    // Act & Assert
    mockMvc.perform(get(uriTemplate:"/api/titulares/lista")
        .param(name:"apellido", apellido)
        .accept(MediaType.APPLICATION_JSON))
        .andExpect(status().isOk())
        .andExpect(jsonPath(expression:"$", hasSize(0)));
}
```



En TitularServiceImplTest:

```
@Test
void buscarTitulares_conFiltroApellido_retornaTitularesFiltrados() {
    // Arrange
    String apellido = "Pérez";

    Titular titular1 = new Titular();
    titular1.setTipoDocumento(TipoDocumento.DNI);
    titular1.setDocumento(documento:"12345678");
    titular1.setNombre(nombre:"Juan");
    titular1.setApellido(apellido:"Pérez");

    Titular titular2 = new Titular();
    titular2.setTipoDocumento(TipoDocumento.DNI);
    titular2.setDocumento(documento:"87654321");
    titular2.setNombre(nombre:"María");
    titular2.setApellido(apellido:"García");

    when(titularRepository.findAll()).thenReturn(List.of(titular1, titular2));

    // Act
    List<Titular> resultado = titularService.buscarTitulares(apellido, tipoDocumento:null, documento:null);

    // Assert
    assertEquals(expected:1, resultado.size());
    assertEquals(expected:"Pérez", resultado.get(index:0).getApellido());
    assertEquals(expected:"Juan", resultado.get(index:0).getNombre());
}

@Test
void buscarTitulares_conFiltroTipoDocumento_retornaTitularesFiltrados() {
    // Arrange
    TipoDocumento tipoDocumento = TipoDocumento.PASAPORTE;

    Titular titular1 = new Titular();
    titular1.setTipoDocumento(TipoDocumento.DNI);
    titular1.setDocumento(documento:"12345678");
    titular1.setNombre(nombre:"Juan");
    titular1.setApellido(apellido:"Pérez");

    Titular titular2 = new Titular();
    titular2.setTipoDocumento(TipoDocumento.PASAPORTE);
    titular2.setDocumento(documento:"AB123456");
    titular2.setNombre(nombre:"Ana");
    titular2.setApellido(apellido:"García");

    when(titularRepository.findAll()).thenReturn(List.of(titular1, titular2));

    // Act
    List<Titular> resultado = titularService.buscarTitulares(apellido:null, tipoDocumento, documento:null);

    // Assert
    assertEquals(expected:1, resultado.size());
    assertEquals(TipoDocumento.PASAPORTE, resultado.get(index:0).getTipoDocumento());
    assertEquals(expected:"AB123456", resultado.get(index:0).getDocumento());
}
```

ID HISTORIA: 11 - ID TAREA: 48

Se implementó la lógica para dar de alta un usuario administrativo. Se agregó la clase SeguridadConfig para poder encriptar las contraseñas con BCrypt. Se realizaron validaciones en el backend de nombres de usuario ya registrados. Se agregó el método registrarUsuario(UsuarioRecord usuarioRecord) al UsuarioService y el método registrarUsuario(UsuarioRecord usuarioRecord) al UsuarioController.



```
@Configuration
public class SeguridadConfig {

    // Bean para codificar contraseñas con BCrypt
    @Bean
    public BCryptPasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    // Configuración de seguridad HTTP
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            // Permitir todas las solicitudes sin autenticación
            .authorizeHttpRequests(auth -> auth.anyRequest().permitAll())
            // Deshabilitar protección CSRF
            .csrf().disable()
            // Deshabilitar formulario de login (sin autenticación)
            .formLogin().disable();

        return http.build();
    }
}
```

```
@PostMapping("/registrar")
public ResponseEntity<?> registrarUsuario(@RequestBody UsuarioRecord usuarioRecord) {
    try {
        usuarioService.registrarUsuario(usuarioRecord);
        return ResponseEntity.ok(Map.of(k1:"message", v1:"Usuario registrado correctamente"));
    } catch (IllegalArgumentException e) {
        return ResponseEntity.badRequest().body(Map.of(k1:"message", e.getMessage()));
    } catch (Exception e) {
        return ResponseEntity.internalServerError().body(Map.of(k1:"message", v1:"Ocurrió un error al registrar el usuario"));
    }
}
```

```
@Override
public void registrarUsuario(UsuarioRecord usuarioRecord) {
    if (repository.existsByNombreUsuario(usuarioRecord.nombreUsuario())) {
        throw new IllegalArgumentException(s,"El nombre de usuario ya está registrado.");
    }

    Usuario usuario = new Usuario();
    usuario.setNombreUsuario(usuarioRecord.nombreUsuario());
    usuario.setTipoDocumento(usuarioRecord.tipoDocumento());
    usuario.setDocumento(usuarioRecord.documento());
    usuario.setApellido(usuarioRecord.apellido());
    usuario.setNombre(usuarioRecord.nombre());
    usuario.setContraseña(passwordEncoder.encode(usuarioRecord.contrasena()));
    usuario.setRol(rol:"Administrativo");

    repository.save(usuario);
}
```

ID HISTORIA: 11 - ID TAREA: 50

Se diseñaron las interfaces para el registro de un usuario administrativo. Una de las interfaces representa el menú para la gestión de usuarios administrativos (donde se incluyen las opciones “Registrar usuario” y “Buscar usuarios”), y la otra el registro de los datos correspondientes (tipo de documento, documento, apellido, nombre, nombre de usuario y contraseña) y sus validaciones. Se conectaron las interfaces con el UsuarioController.

```
@GetMapping("/menuUsuario")
String mostrarMenuUsuario() {
    return "menuUsuarioAdmin";
}

@GetMapping("/registroUsuarioAdministrativo")
String mostrarRegistroUsuarioAdministrativo() {
    return "registroUsuario";
}
```



localhost:8080 dice
Por favor, complete todos los campos obligatorios.

Aceptar

Seleccione...

Este campo es obligatorio.

Número de documento

Este campo es obligatorio.

Apellido/s *

Este campo es obligatorio.

Nombre/s *

Este campo es obligatorio.

Nombre de usuario *

Este campo es obligatorio.

Contraseña *

Este campo es obligatorio.

Confirmar contraseña *

Este campo es obligatorio.

Registrar usuario

Registrar usuario

Tipo de documento

DNI

Número de documento

12345678

Apellido/s

Lopez

Nombre/s

Marcos

Nombre de usuario *

marcos05

El nombre de usuario ya está registrado.

Contraseña *

.....

Confirmar contraseña *

.....

Registrar usuario



ID TAREA: 11 - ID TAREA: 49

Se realizaron tests de la lógica de creación de usuarios administrativos. Para ello se crearon las clases UsuarioControllerTest y UsuarioServiceTest.

En la clase UsuarioControllerTest se desarrollaron los siguientes tests:

- registrarUsuario_exitoso()
- registrarUsuario_nombreUsuarioRepetido()

En la clase UsuarioServiceTest se desarrollaron los siguientes tests:

- registrarUsuario_nombreUsuarioYaExiste_lanzaException()
- registrarUsuario_datosValidos_guardarUsuario()

```
@Test
void registrarUsuario_exitoso() throws Exception {
    Mockito.doNothing().when(usuarioService).registrarUsuario(any());

    mockMvc.perform(post(uriTemplate:"/api/usuarios/registrar")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(usuarioRecord)))
        .andExpect(status().isOk())
        .andExpect(jsonPath(expression:"$.message").value(expectedValue:"Usuario registrado correctamente"));
}

@Test
void registrarUsuario_nombreUsuarioRepetido() throws Exception {
    Mockito.doThrow(new IllegalArgumentException(s:"El nombre de usuario ya está registrado.")).when(usuarioService).registrarUsuario(any());

    mockMvc.perform(post(uriTemplate:"/api/usuarios/registrar")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(usuarioRecord)))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath(expression:"$.message").value(expectedValue:"El nombre de usuario ya está registrado."));
}
```

```
@Test
void registrarUsuario_nombreUsuarioYaExiste_lanzaException() {
    UsuarioRecord usuarioRecord = new UsuarioRecord(nombreUsuario:"admin", TipoDocumento.DNI, documento:"12345678", apellido:"Perez", nombre:"Juan", con..."clave123");

    when(repository.existsByNombreUsuario(nombreUsuario:"admin")).thenReturn(value:true);

    IllegalArgumentException ex = assertThrows(expectedType:IllegalArgumentException.class,
        () -> usuarioService.registrarUsuario(usuarioRecord));

    assertEquals(expected:"El nombre de usuario ya está registrado.", ex.getMessage());
}

@Test
void registrarUsuario_datosValidos_guardarUsuario() {
    UsuarioRecord usuarioRecord = new UsuarioRecord(nombreUsuario:"admin", TipoDocumento.DNI, documento:"12345678", apellido:"Perez", nombre:"Juan", con..."clave123");

    when(repository.existsByNombreUsuario(nombreUsuario:"admin")).thenReturn(value:false);
    when(passwordEncoder.encode("clave123")).thenReturn(value:"hashedPassword");

    usuarioService.registrarUsuario(usuarioRecord);

    verify(repository, times(wantedNumberOfInvocations:1)).save(argThat(usuario ->
        usuario.getNombreUsuario().equals(anObject:"admin") &&
        usuario.getDocumento().equals(anObject:"12345678") &&
        usuario.getApellido().equals(anObject:"Perez") &&
        usuario.getNombre().equals(anObject:"Juan") &&
        usuario.getContraseña().equals(anObject:"hashedPassword") &&
        usuario.getRol().equals(anObject:"Administrativo")
    ));
}
```



ID HISTORIA: 13 - ID TAREA: 61

Se crearon las interfaces **menuOpcionesUsuarioAdministrativo.html** y **menuOpcionesUsuarioAdministrador.html**. La primera interfaz contiene las opciones “Gestión de titulares” y “Gestión de licencias”, y la segunda interfaz contiene las opciones “Gestión de usuarios”, “Gestión de titulares” y “Gestión de licencias”. Se conectaron ambas interfaces con las demás.

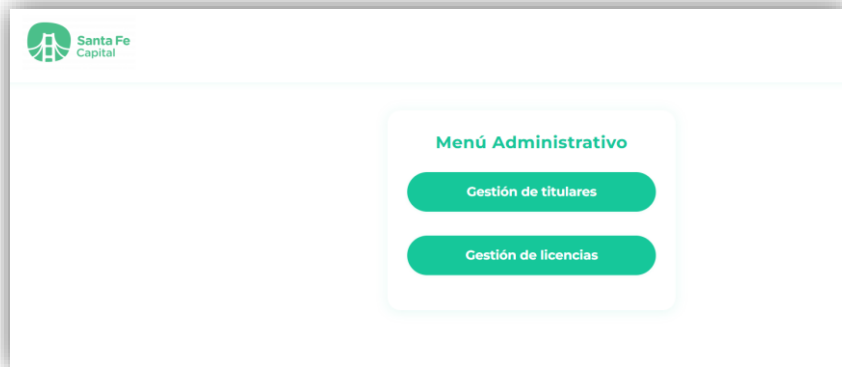
Se creó el UsuarioController y se conectaron las interfaces con el mismo.

```
@Tag(name = "Usuario Controller", description = "Operaciones para la gestión de usuarios")
@Controller
@RequestMapping("/api/usuarios")
public class UsuarioController {

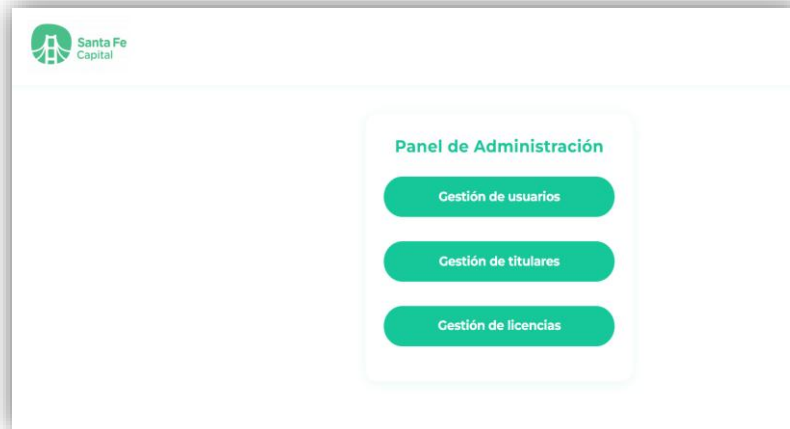
    @GetMapping("/menuOpcionesAdministrativo")
    String mostrarMenuOpcionesAdministrativo(){
        return "menuOpcionesUsuarioAdministrativo";
    }

    @GetMapping("/menuOpcionesAdministrador")
    String mostrarMenuOpcionesAdministrador(){
        return "menuOpcionesUsuarioAdministrador";
    }
}
```

```
<div class="container card-menu">
  <h2><b>Menú Administrativo</b></h2>
  <div class="d-grid gap-3">
    <a href="/api/titulares" class="btn btn-menu">Gestión de titulares</a>
    <a href="/api/licencias" class="btn btn-menu">Gestión de licencias</a>
  </div>
```



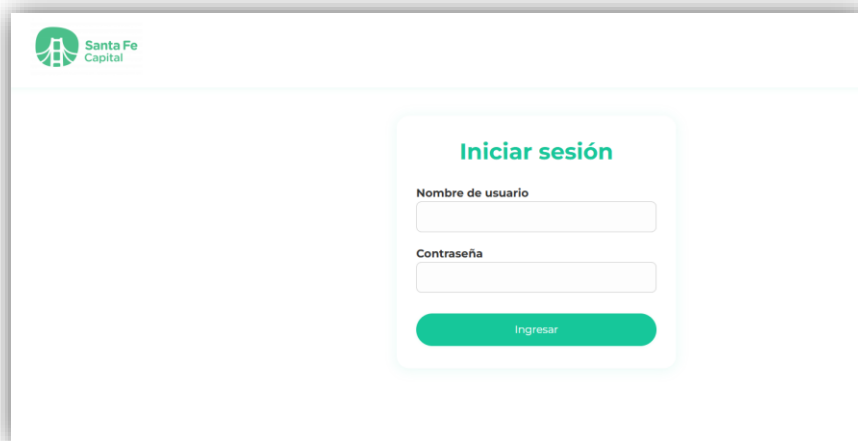
```
<div class="container card-menu">
  <h2><b>Panel de Administración</b></h2>
  <div class="d-grid gap-3">
    <a href="/api/usuarios" class="btn btn-menu">Gestión de usuarios</a>
    <a href="/api/titulares" class="btn btn-menu">Gestión de titulares</a>
    <a href="/api/licencias" class="btn btn-menu">Gestión de licencias</a>
  </div>
```



ID HISTORIA: 13 - ID TAREA: 46

Se desarrolló la interfaz de login para usuarios administradores y administrativos. En la misma se requiere nombre de usuario y contraseña.

Se agregó el método mostrarLogin() al UsuarioController.



```
@GetMapping("/login")
String mostrarLogin() {
    return "login";
}
```

ID TAREA: 11 - ID HISTORIA: 47

Se desarrolló la lógica para el inicio de sesión de usuarios administradores y administrativos. Según quien inicia sesión, se muestra un menú de opciones u otro.



Se agregó el método loginUsuario(String username, String password, Model model, RedirectAttributes redirectAttributes) en el UsuarioController, y el método loginDeUsuario(String username, String password) en el UsuarioService.

```
@GetMapping("/login")
String mostrarLogin() {
    return "login";
}

@Operation(
    summary = "Login de Usuario",
    description = "Permite a un usuario iniciar sesión con su nombre de usuario y contraseña.",
    responses = {
        @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "200", description = "Login exitoso"),
        @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "401", description = "Credenciales inválidas"),
        @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "500", description = "Error interno del servidor")
    }
)
@PostMapping("/loginInput")
public String loginUsuario(
    @RequestParam String username,
    @RequestParam String password,
    Model model,
    RedirectAttributes redirectAttributes) {

    try {
        String token = usuarioService.loginDeUsuario(username, password);
        String rol = JwtTokenParser(
            .setSigningKey(jwtSecretKey)
            .parseClaimsJws(token)
            .getBody()
            .get("rol", String.class);

        return "root".equals(rol)
            ? "redirect:/api/usuarios/Administrador"
            : "redirect:/api/usuarios/Administrativo";
    } catch (IllegalArgumentException e) {
        redirectAttributes.addAttribute("error", "true");
        return "redirect:/api/usuarios/login";
    }
}
```

```
@Override
public String loginDeUsuario(String username, String password) {

    Usuario usuario = repository.findByNombreUsuario(username);

    if (usuario == null) {
        throw new IllegalArgumentException(s:"Usuario no encontrado.");
    }

    if (!passwordEncoder.matches(password, usuario.getContraseña())) {
        throw new IllegalArgumentException(s:"Contraseña incorrecta.");
    }

    String token = JwtTokenBuilder(
        .setSubject(username)
        .claim("rol", usuario.getRol())
        .setIssuedAt(new Date())
        .setExpiration(new Date(System.currentTimeMillis() + 3600000))
        .signWith(SignatureAlgorithm.HS256, jwtSecretKey)
        .compact();

    return token;
}
```

ID HISTORIA: 13 - ID TAREA: 59

Se realizaron pruebas de seguridad en el inicio de sesión en el sistema. Se adjuntan algunos.

En la clase UsuarioServiceTest:



```

@Test
void loginDeUsuario_credencialesValidas_retornaToken() {
    // Arrange
    String username = "admin";
    String password = "password123";
    String hashedPassword = "$2a$10$hashedPassword";

    Usuario usuario = new Usuario();
    usuario.setNombreUsuario(username);
    usuario.setContraseña(hashedPassword);
    usuario.setRol(rol:"Administrativo");

    when(repository.findByNombreUsuario(username)).thenReturn(usuario);
    when(passwordEncoder.matches(password, hashedPassword)).thenReturn(value:true);

    // Act
    String token = usuarioService.loginDeUsuario(username, password);

    // Assert
    assertNotNull(token);
    assertTrue(token.length() > 0);
    verify(repository).findByNombreUsuario(username);
    verify(passwordEncoder).matches(password, hashedPassword);
}

@Test
void loginDeUsuario_contraseñaIncorrecta_lanzaException() {
    // Arrange
    String username = "admin";
    String password = "contraseña_incorrecta";
    String hashedPassword = "$2a$10$hashedPassword";

    Usuario usuario = new Usuario();
    usuario.setNombreUsuario(username);
    usuario.setContraseña(hashedPassword);
    usuario.setRol(rol:"Administrativo");

    when(repository.findByNombreUsuario(username)).thenReturn(usuario);
    when(passwordEncoder.matches(password, hashedPassword)).thenReturn(value:false);

    // Act & Assert
    IllegalArgumentException exception = assertThrows(
        expectedType: IllegalArgumentException.class,
        () -> usuarioService.loginDeUsuario(username, password),
        message:"Debería lanzar excepción cuando la contraseña es incorrecta"
    );

    assertEquals(expected:"Contraseña incorrecta.", exception.getMessage());
    verify(repository).findByNombreUsuario(username);
    verify(passwordEncoder).matches(password, hashedPassword);
}

```

En UsuarioControllerTest:

```
@Test
void loginUsuario_credencialesValidas_redirigeSegunRol() throws Exception {
    // Arrange
    String username = "admin";
    String password = "password123";
    String token = "eyJhbGciOiJIUzI1NiJ9.eyJzdWUiOiJhZG1pbSIzInJvcCI6IkFkbWluaXN0cmF0aXZvIiwiaWF0IjoxNjM5NzI5NjAwLCJleHAiOiJlMzZkMzMyMDB9.signature";

    when(usuarioService.loginDeUsuario(username, password)).thenReturn(token);

    // Act & Assert
    mockMvc.perform(post(uriTemplate:"/api/usuarios/loginInput")
        .param(name:"username", username)
        .param(name:"password", password)
        .contentType(MediaType.APPLICATION_FORM_URLENCODED))
        .andExpect(status().is3xxRedirection())
        .andExpect(redirectedUrl(expectedUrl:"/api/usuarios/Administrativo"));
}

@Test
void loginUsuario_credencialesInvalidas_redirigeALoginConError() throws Exception {
    // Arrange
    String username = "usuario_inexistente";
    String password = "password123";

    when(usuarioService.loginDeUsuario(username, password))
        .thenReturn(new IllegalArgumentException(s:"Usuario no encontrado."));

    // Act & Assert
    mockMvc.perform(post(uriTemplate:"/api/usuarios/loginInput")
        .param(name:"username", username)
        .param(name:"password", password)
        .contentType(MediaType.APPLICATION_FORM_URLENCODED))
        .andExpect(status().is3xxRedirection())
        .andExpect(redirectedUrl(expectedUrl:"/api/usuarios/login?error=true"));
}
```

ID HISTORIA: 12 - ID TAREA: 65

Se implementó la lógica de negocio que permite buscar usuarios administrativos registrados en el sistema en función de distintos criterios. La lógica permite realizar búsquedas aplicando uno o más de los siguientes filtros:

- Nombre de usuario (username o identificador único del sistema).
- Tipo de documento (por ejemplo, DNI, pasaporte).
- Número de documento.

```
1 @GetMapping("/buscar")
2 public ResponseEntity<List<UsuarioRecord>> buscarUsuario(
3     @RequestParam(required = false) TipoDocumento tipoDocumento,
4     @RequestParam(required = false) String documento,
5     @RequestParam(required = false) String nombreUsuario) {
6     try{
7         List<UsuarioRecord> usuarios = usuarioService.buscarUsuario(tipoDocumento, documento, nombreUsuario);
8         return ResponseEntity.ok(usuarios);
9     } catch (IllegalArgumentException e){
10         return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
11     }
12 }
```

```
1 @Override
2 public List<UsuarioRecord> buscarUsuario(TipoDocumento tipoDocumento, String documento, String nombreUsuario) {
3     List<Usuario> usuarios = repository.buscarPorFiltros(tipoDocumento,
4         (documento != null && !documento.isEmpty()) ? documento : null,
5         (nombreUsuario != null && !nombreUsuario.isEmpty()) ? nombreUsuario : null);
6     return usuarios.stream()
7         .map(usuario -> new UsuarioRecord(
8             usuario.getNombreUsuario(),
9             usuario.getTipoDocumento(),
10            usuario.getDocumento(),
11            usuario.getApellido(),
12            usuario.getNombre(),
13            usuario.getContraseña()
14        ))
15         .toList();
16 }
```

```
1 @Query("SELECT u FROM Usuario u WHERE " +
2     "(:tipoDocumento IS NULL OR u.tipoDocumento = :tipoDocumento) AND " +
3     "(:documento IS NULL OR u.documento = :documento) AND " +
4     "(:nombreUsuario IS NULL OR u.nombreUsuario = :nombreUsuario)")
5 List<Usuario> buscarPorFiltros(@Param("tipoDocumento") TipoDocumento tipoDocumento,
6     @Param("documento") String documento,
7     @Param("nombreUsuario") String nombreUsuario);
8 }
```

Comentario: Empecé a las 9:00 hs y terminé a las 11:00 hs. Es decir, me llevó 2 hs aproximadamente, por lo tanto se cumplió con el objetivo ya que el task point definido era de 2.

ID HISTORIA: 12 - ID TAREA: 55

Se implementó la lógica de negocio que permite actualizar los datos personales de un usuario administrativo registrado en el sistema.

La lógica contempla los siguientes requerimientos funcionales establecidos en la historia de usuario:

- Recepción de criterios de búsqueda (nombre de usuario, tipo de documento y número de documento) para localizar al usuario a modificar (estos son optativos).
- Validación de existencia del usuario solicitado antes de proceder a la modificación.
- Obtención y exposición de los datos actuales del usuario encontrado, con el fin de mostrarlos en la interfaz correspondiente para su edición.
- Modificación de los campos editables, que incluyen: nombre, apellido, contraseña y demás datos personales definidos en el modelo.
- Persistencia de los cambios en la base de datos, asegurando la integridad de la información.
- Verificación de permisos: únicamente los usuarios con rol administrador deben tener autorización para ejecutar esta operación.

```
1 @PostMapping("/modificar")
2 public ResponseEntity<?> modificarUsuario(@RequestBody UsuarioRecord usuarioRecord) {
3     try {
4         usuarioService.modificarUsuario(usuarioRecord);
5         return ResponseEntity.ok(Map.of("message", "Usuario modificado correctamente"));
6     } catch (IllegalArgumentException e) {
7         return ResponseEntity.badRequest().body(Map.of("message", e.getMessage()));
8     } catch (Exception e) {
9         return ResponseEntity.internalServerError().body(Map.of("message", "Ocurrió un error al modificar el usuario"));
10    }
11 }
```

```
@Override
public void modificarUsuario(UsuarioRecord usuarioRecord) {
    Usuario usuario = repository.findByTipoDocumentoAndDocumento(usuarioRecord.tipoDocumento(), usuarioRecord.documento());
    if (usuario == null) {
        throw new IllegalArgumentException("Usuario no encontrado.");
    }

    usuario.setApellido(usuarioRecord.apellido());
    usuario.setNombre(usuarioRecord.nombre());
    if (usuarioRecord.contrasena() != null && !usuarioRecord.contrasena().isBlank()) {
        usuario.setContrasena(passwordEncoder.encode(usuarioRecord.contrasena()));
    }

    repository.save(usuario);
}
```

Comentario: Empecé a las 11:00 hs y terminé a las 12:00 hs. Es decir, me llevó 1 h. aproximadamente, por lo que se sobre cumplió con los tasks point estimados para esta tarea.

ID HISTORIA: 12 - ID TAREA: 56

Se desarrolló la interfaz para buscar usuarios administrativos por tipo de documento, número de documento y/o nombre de usuario. También se desarrolló la interfaz para modificar un usuario administrativo, la cual requiere los mismos campos que el registro. Se encuentran bloqueados los



campos tipo de documento, número de documento y nombre de usuario. Se realizaron las validaciones correspondientes. Se conectaron las interfaces con el UsuarioController, donde se agregaron los métodos `mostrarBusquedaUsuarios()` y `mostrarFormularioModificacion(TipoDocumento tipoDocumento, String documento, String nombreUsuario, Model model)`.

```
@GetMapping("/buscarUsuarios")
public String mostrarBusquedaUsuarios() {
    return "listaUsuarios";
}

@Operation(
    summary = "Obtener usuario por tipo y documento",
    description = "Devuelve un usuario según tipo de documento y número de documento",
    responses = {
        @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "200", description = "Usuario encontrado"),
        @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "404", description = "Usuario no encontrado"),
        @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "400", description = "Parámetros inválidos"),
        @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "500", description = "Error interno del servidor")
    }
)
@GetMapping("/modificar/{tipoDocumento}/{documento}/{nombreUsuario}")
public String mostrarFormularioModificacion(
    @PathVariable TipoDocumento tipoDocumento,
    @PathVariable String documento,
    @PathVariable String nombreUsuario,
    Model model) {

    List<UsuarioRecord> usuarios = usuarioService.buscarUsuario(tipoDocumento, documento, nombreUsuario);
    model.addAttribute(attributeName:"usuario", usuarios.get(index:0));

    model.addAttribute(attributeName:"tiposDocumento", TipoDocumento.values());

    return "modificacionUsuario";
}
```


Santa Fe Capital

Buscar usuarios

Tipo de documento

Todos

Documento

Buscar por número de documento

Nombre de usuario

marcos05

Buscar

Usuario	Tipo Doc	Documento	Apellido	Nombre
marcos05	DNI	12345678	Lopez	Marcos

Modificar usuario



**Santa Fe
Capital**

localhost:8080 dice
No se encontraron usuarios.
[Aceptar](#)

Buscar usuarios

Tipo de documento
Todos

Documento
Buscar por número de documento

Nombre de usuario
pedro01

Buscar

Modificar usuario

Modificar usuario

Tipo de documento
DNI

Número de documento
12345678

Nombre de usuario
marcos05

Apellido
Lopez

Nombre
Marcos

Contraseña nueva

Confirmar contraseña

Guardar cambios



localhost:8080 dice
Usuario modificado correctamente.

Aceptar

DNI

Número de documento
12345678

Nombre de usuario
marcos05

Apellido
Lopez

Nombre
Marcos

Contraseña nueva

Confirmar contraseña

Guardar cambios

localhost:8080 dice
Por favor, complete todos los campos obligatorios correctamente.

Aceptar

Número de documento
12345678

Nombre de usuario
marcos05

Apellido
Lopez

Nombre
Marcos44
El nombre sólo puede contener letras y espacios.

Contraseña nueva

Confirmar contraseña

Las contraseñas no coinciden.

Guardar cambios

ID HISTORIA: 12 - ID TAREA: 57

Se diseñaron e implementaron pruebas unitarias que validan el correcto funcionamiento de la lógica de búsqueda y modificación de usuarios administrativos, asegurando que los casos previstos por la historia



de usuario se comporten conforme a lo esperado. Las pruebas contemplan escenarios exitosos y casos de error, y sirven como respaldo para garantizar la calidad y estabilidad del sistema ante futuros cambios.

Se adjuntan solo algunos de los test unitarios, puesto que se realizaron para varios casos.

```
1  @Test
2  void buscarUsuario_exitoso() throws Exception {
3      UsuarioRecord usuario1 = new UsuarioRecord("admin", TipoDocumento.DNI, "12345678", "Perez", "Juan", "clave123");
4      UsuarioRecord usuario2 = new UsuarioRecord("ana01", TipoDocumento.DNI, "87654321", "Gomez", "Ana", "clave456");
5      when(usuarioService.buscarUsuario(null, null, null)).thenReturn(List.of(usuario1, usuario2));
6
7      mockMvc.perform(get("/api/usuarios/buscar"))
8          .andExpect(status().isOk())
9          .andExpect(jsonPath("$.nombreUsuario").value("admin"))
10         .andExpect(jsonPath("$.apellido").value("Juan"));
11 }
12
13 @Test
14 void buscarUsuario_sinResultados() throws Exception {
15     when(usuarioService.buscarUsuario(TipoDocumento.DNI, "99999999", null)).thenReturn(List.of());
16
17     mockMvc.perform(get("/api/usuarios/buscar")
18         .param("tipoDocumento", "DNI")
19         .param("documento", "99999999"))
20         .andExpect(status().isOk())
21         .andExpect(jsonPath("$.nombreUsuario").isEmpty());
22 }
```

```
1  @Test
2  void buscarUsuario_todosNulos_devuelveTodos() {
3      var usuario1 = new com.gestionlicencias.gestionlicenciasconducir.model.Usuario();
4      usuario1.setNombreUsuario("admin");
5      usuario1.setTipoDocumento(TipoDocumento.DNI);
6      usuario1.setDocumento("12345678");
7      usuario1.setApellido("Perez");
8      usuario1.setNombre("Juan");
9      usuario1.setContraseña("clave123");
10     usuario1.setRol("Administrativo");
11
12     var usuario2 = new com.gestionlicencias.gestionlicenciasconducir.model.Usuario();
13     usuario2.setNombreUsuario("ana01");
14     usuario2.setTipoDocumento(TipoDocumento.DNI);
15     usuario2.setDocumento("87654321");
16     usuario2.setApellido("Gomez");
17     usuario2.setNombre("Ana");
18     usuario2.setContraseña("clave456");
19     usuario2.setRol("Administrativo");
20 }
```




```

1  @Test
2  void modificarUsuario_usuarioExistente_modificaDatos() {
3      UsuarioRecord usuarioRecord = new UsuarioRecord("admin", TipoDocumento.DNI, "12345678", "NuevoApellido", "NuevoNombre", "nuevaClave");
4      com.gestionlicencias.gestionlicenciasconducir.model.Usuario usuario = new com.gestionlicencias.gestionlicenciasconducir.model.Usuario();
5      usuario.setNombreUsuario("admin");
6      usuario.setTipoDocumento(TipoDocumento.DNI);
7      usuario.setDocumento("12345678");
8      usuario.setApellido("ViejoApellido");
9      usuario.setNombre("ViejoNombre");
10     usuario.setContraseña("viejaClave");
11     usuario.setRol("Administrativo");
12
13     when(repository.findByTipoDocumentoAndDocumento(TipoDocumento.DNI, "12345678")).thenReturn(usuario);
14     when(passwordEncoder.encode("nuevaClave")).thenReturn("hashedNuevaClave");
15
16     usuarioService.modificarUsuario(usuarioRecord);
17
18     assertEquals("NuevoApellido", usuario.getApellido());
19     assertEquals("NuevoNombre", usuario.getNombre());
20     assertEquals("hashedNuevaClave", usuario.getContraseña());
21     verify(repository).save(usuario);
22 }
23
24 @Test
25 void modificarUsuario_usuarioNoExiste_lanzaException() {
26     UsuarioRecord usuarioRecord = new UsuarioRecord("admin", TipoDocumento.DNI, "12345678", "Apellido", "Nombre", "clave");
27     when(repository.findByTipoDocumentoAndDocumento(TipoDocumento.DNI, "12345678")).thenReturn(null);
28
29     IllegalArgumentException ex = assertThrows(IllegalArgumentException.class,
30         () -> usuarioService.modificarUsuario(usuarioRecord));
31     assertEquals("Usuario no encontrado.", ex.getMessage());
32     verify(repository, never()).save(any());
33 }

```

Comentarios: Empecé a las 12:00 hs y terminé a las 12:15 hs. Es decir, me llevó 15 minutos aproximadamente, por lo tanto, se sobre cumplió el objetivo.

DAILY SCRUM

Daily Meeting 1 - 13/06/2025.

Asistentes: equipo completo.

Temas tratados:

Progreso:

- Desarrollo de la funcionalidad completa para la búsqueda de licencias vigentes (ID Tareas 37, 38, 39, 62).

Desvíos:

- Ninguno reportado.

Acuerdos:

- Se propuso avanzar con las tareas de licencias vencidas y renovación durante la semana.

Duración: 30 minutos.

**Estado del sprint backlog:**

ID Historia	ID Tarea	Nombre tarea	Dueño	Esfuerzo	Estado
8	37	Diseñar interfaz para la búsqueda de licencias vigentes	Aldana	1	Finalizado
8	38	Implementar lógica de búsqueda de licencias vigentes por criterios	Aldana	2	Finalizado
8	39	Realizar pruebas de funcionalidad de búsqueda de licencias	Aldana	2	Finalizado
8	62	Diseñar interfaz de menú de licencias	Aldana	1	Finalizado
7	40	Desarrollar lógica para listar licencias expiradas	Santiago	4	En proceso
7	41	Diseñar interfaz de listado de licencias expiradas	Franco	2	En proceso
7	42	Realizar pruebas de listado de licencias expiradas	Santiago	1	Pendiente
5	25	Diseñar interfaz de renovación de licencia	Aldana	1	Pendiente
5	26	Desarrollar lógica de renovación de licencias	Aldana	2	Pendiente
5	27	Realizar pruebas de renovación de licencias	Victoria	2	Pendiente
6	43	Desarrollar la funcionalidad para solicitar copia de una licencia	Aldana	2	Pendiente
6	45	Realizar pruebas de emisión de copias de licencias	Franco	1	Pendiente
10	52	Diseñar interfaces para modificar un titular con sus validaciones	Victoria	1	Pendiente
10	54	Desarrollar lógica de modificación de datos de un titular	Aldana	1	Finalizado
10	63	Desarrollar lógica para buscar titulares	Franco	2	Pendiente
10	58	Realizar pruebas de búsqueda y modificación de titulares	Lara	2	Pendiente
11	48	Implementar lógica de registro de un usuario administrativo	Franco	2	Pendiente
11	50	Diseñar interfaces para el registro de un usuario administrativo con sus validaciones	Victoria	1	Pendiente
11	49	Realizar pruebas de creación de usuarios administrativos	Lara	1	Pendiente
13	46	Diseñar interfaz para login de usuarios (administradores y administrativos)	Aldana	1	En proceso
13	47	Desarrollar lógica para login de usuarios	Franco	2	Pendiente
13	59	Realizar pruebas de seguridad	Lara	2	Pendiente
13	61	Diseñar interfaces de menú para usuarios administradores y administrativos	Aldana	1	Finalizado
12	55	Desarrollar lógica para modificar un usuario administrativo	Santiago	2	Pendiente
12	56	Diseñar interfaces para modificar datos de un usuario administrativo con sus validaciones	Aldana	1	Pendiente
12	65	Desarrollar lógica para buscar usuarios administrativos	Santiago	2	Pendiente
12	57	Realizar pruebas de búsqueda y modificación de usuarios administrativos	Santiago	1	Pendiente

Tabla 9 – Estado del sprint backlog en daily meeting 1**Daily Meeting 2 - 20/06/2025.****Asistentes:** Aldana, Santiago, Franco.**Temas tratados:****Progreso:**

- Desarrollo de lógica para listar licencias expiradas (ID Tarea 40).
- Diseño de interfaz del listado de licencias expiradas (ID Tarea 41).
- Interfaz de renovación de licencia (ID Tarea 25).
- Desarrollo de lógica para renovación de licencias (ID Tarea 26).

Desvíos:

- La interfaz de expiradas llevó el doble del tiempo estimado por ajustes visuales.

Acuerdos:

- Avanzar con desarrollo de renovación de licencias y solicitud de copias de licencias, con sus respectivas pruebas.
- Terminar tareas pendientes sobre gestión de usuarios administrativos.



Duración: 40 minutos.

Estado del sprint backlog:

ID Historia	ID Tarea	Nombre tarea	Dueño	Esfuerzo	Estado
8	37	Diseñar interfaz para la búsqueda de licencias vigentes	Aldana	1	Finalizado
8	38	Implementar lógica de búsqueda de licencias vigentes por criterios	Aldana	2	Finalizado
8	39	Realizar pruebas de funcionalidad de búsqueda de licencias	Aldana	2	Finalizado
8	62	Diseñar interfaz de menú de licencias	Aldana	1	Finalizado
7	40	Desarrollar lógica para listar licencias expiradas	Santiago	4	Finalizado
7	41	Diseñar interfaz de listado de licencias expiradas	Franco	2	Finalizado
7	42	Realizar pruebas de listado de licencias expiradas	Santiago	1	Pendiente
5	25	Diseñar interfaz de renovación de licencia	Aldana	1	En proceso
5	26	Desarrollar lógica de renovación de licencias	Aldana	2	En proceso
5	27	Realizar pruebas de renovación de licencias	Victoria	2	Pendiente
6	43	Desarrollar la funcionalidad para solicitar copia de una licencia	Aldana	2	Pendiente
6	45	Realizar pruebas de emisión de copias de licencias	Franco	1	Pendiente
10	52	Diseñar interfaces para modificar un titular con sus validaciones	Victoria	1	Pendiente
10	54	Desarrollar lógica de modificación de datos de un titular	Aldana	1	Finalizado
10	63	Desarrollar lógica para buscar titulares	Franco	2	Pendiente
10	58	Realizar pruebas de búsqueda y modificación de titulares	Lara	2	Pendiente
11	48	Implementar lógica de registro de un usuario administrativo	Franco	2	En proceso
11	50	Diseñar interfaces para el registro de un usuario administrativo con sus validaciones	Victoria	1	En proceso
11	49	Realizar pruebas de creación de usuarios administrativos	Lara	1	Pendiente
13	46	Diseñar interfaz para login de usuarios (administradores y administrativos)	Aldana	1	Finalizado
13	47	Desarrollar lógica para login de usuarios	Franco	2	Pendiente
13	59	Realizar pruebas de seguridad	Lara	2	Pendiente
13	61	Diseñar interfaces de menú para usuarios administradores y administrativos	Aldana	1	Finalizado
12	55	Desarrollar lógica para modificar un usuario administrativo	Santiago	2	Pendiente
12	56	Diseñar interfaces para modificar datos de un usuario administrativo con sus validaciones	Aldana	1	Pendiente
12	65	Desarrollar lógica para buscar usuarios administrativos	Santiago	2	Pendiente
12	57	Realizar pruebas de búsqueda y modificación de usuarios administrativos	Santiago	1	Pendiente

Tabla 10 – Estado del sprint backlog en daily meeting 2

Daily Meeting 3 - 27/06/2025.

Asistentes: equipo completo.

Temas tratados:

Progreso:

- Desarrollo completo de módulo de renovación de licencias (ID Tareas 25, 26, 27).
- Desarrollo completo de módulo de solicitudes de copias de licencias (ID Tareas 43, 45).
- Finalización de tareas pendientes.

Desvíos:

- Se reportaron algunas complicaciones en el desarrollo del módulo de solicitudes de copias de licencias (conexión entre interfaces, registro del número de copia solicitada, entre otros).

Acuerdos:



- Añadir tareas de refactorización para mejorar la búsqueda de licencias vigentes, renovación de licencias y solicitudes de copias (ID Tareas 66, 67, 68).
- Finalizar las tareas en desarrollo. Comenzar y terminar lo antes posible las tareas pendientes.

Duración: 45 minutos.

Estado del sprint backlog:

ID Historia	ID Tarea	Nombre tarea	Dueño	Esfuerzo	Estado
8	37	Diseñar interfaz para la búsqueda de licencias vigentes	Aldana	1	Finalizado
8	38	Implementar lógica de búsqueda de licencias vigentes por criterios	Aldana	2	Finalizado
8	39	Realizar pruebas de funcionalidad de búsqueda de licencias	Aldana	2	Finalizado
8	62	Diseñar interfaz de menú de licencias	Aldana	1	Finalizado
7	40	Desarrollar lógica para listar licencias expiradas	Santiago	4	Finalizado
7	41	Diseñar interfaz de listado de licencias expiradas	Franco	2	Finalizado
7	42	Realizar pruebas de listado de licencias expiradas	Santiago	1	Pendiente
5	25	Diseñar interfaz de renovación de licencia	Aldana	1	Finalizado
5	26	Desarrollar lógica de renovación de licencias	Aldana	2	Finalizado
5	27	Realizar pruebas de renovación de licencias	Victoria	2	Finalizado
6	43	Desarrollar la funcionalidad para solicitar copia de una licencia	Aldana	2	Finalizado
6	45	Realizar pruebas de emisión de copias de licencias	Franco	1	Finalizado
10	52	Diseñar interfaces para modificar un titular con sus validaciones	Victoria	1	Pendiente
10	54	Desarrollar lógica de modificación de datos de un titular	Aldana	1	Finalizado
10	63	Desarrollar lógica para buscar titulares	Franco	2	Pendiente
10	58	Realizar pruebas de búsqueda y modificación de titulares	Lara	2	Pendiente
11	48	Implementar lógica de registro de un usuario administrativo	Franco	2	Finalizado
11	50	Diseñar interfaces para el registro de un usuario administrativo con sus validaciones	Victoria	1	Finalizado
11	49	Realizar pruebas de creación de usuarios administrativos	Lara	1	Pendiente
13	46	Diseñar interfaz para login de usuarios (administradores y administrativos)	Aldana	1	Finalizado
13	47	Desarrollar lógica para login de usuarios	Franco	2	Pendiente
13	59	Realizar pruebas de seguridad	Lara	2	Pendiente
13	61	Diseñar interfaces de menú para usuarios administradores y administrativos	Aldana	1	Finalizado
12	55	Desarrollar lógica para modificar un usuario administrativo	Santiago	2	Pendiente
12	56	Diseñar interfaces para modificar datos de un usuario administrativo con sus validaciones	Aldana	1	Pendiente
12	65	Desarrollar lógica para buscar usuarios administrativos	Santiago	2	Pendiente
12	57	Realizar pruebas de búsqueda y modificación de usuarios administrativos	Santiago	1	Pendiente

Tabla 11 – Estado del sprint backlog en daily meeting 3

Daily Meeting 4 - 04/07/2025.

Asistentes: equipo completo.

Temas tratados:

Progreso:

- Finalización de las tareas pendientes y en proceso.
- Refactorizaciones de funcionalidades críticas: buscarLicenciasVigentes (ID Tarea 66), renovarLicencia (ID Tarea 67), emitirCopiaLicencia (ID Tarea 68).

Desvíos:



- La lógica de login requirió más esfuerzo del previsto (doble del estimado) por desconocimiento de las herramientas utilizadas.

Acuerdos:

- Revisar oportunidades de simplificación en lógica de autenticación.

Duración: 50 minutos.

Estado del product backlog:

ID Historia	ID Tarea	Nombre tarea	Dueño	Esfuerzo	Estado
8	37	Diseñar interfaz para la búsqueda de licencias vigentes	Aldana	1	Finalizado
8	38	Implementar lógica de búsqueda de licencias vigentes por criterios	Aldana	2	Finalizado
8	39	Realizar pruebas de funcionalidad de búsqueda de licencias	Aldana	2	Finalizado
8	62	Diseñar interfaz de menú de licencias	Aldana	1	Finalizado
8	66	Refactorizar "buscarLicenciasVigentes".	Aldana	1	Finalizado
7	40	Desarrollar lógica para listar licencias expiradas	Santiago	4	Finalizado
7	41	Diseñar interfaz de listado de licencias expiradas	Franco	2	Finalizado
7	42	Realizar pruebas de listado de licencias expiradas	Santiago	1	Finalizado
5	25	Diseñar interfaz de renovación de licencia	Aldana	1	Finalizado
5	26	Desarrollar lógica de renovación de licencias	Aldana	2	Finalizado
5	27	Realizar pruebas de renovación de licencias	Victoria	2	Finalizado
5	67	Refactorizar "renovarLicencia"	Victoria	1	Finalizado
6	43	Desarrollar la funcionalidad para solicitar copia de una licencia	Aldana	2	Finalizado
6	45	Realizar pruebas de emisión de copias de licencias	Franco	1	Finalizado
6	68	Refactorizar "emitirCopiaLicencia"	Franco	1	Finalizado
10	52	Diseñar interfaces para modificar un titular con sus validaciones	Victoria	1	Finalizado
10	54	Desarrollar lógica de modificación de datos de un titular	Aldana	1	Finalizado
10	63	Desarrollar lógica para buscar titulares	Franco	2	Finalizado
10	58	Realizar pruebas de búsqueda y modificación de titulares	Lara	2	Finalizado
11	48	Implementar lógica de registro de un usuario administrativo	Franco	2	Finalizado
11	50	Diseñar interfaces para el registro de un usuario administrativo con sus validaciones	Victoria	1	Finalizado
11	49	Realizar pruebas de creación de usuarios administrativos	Lara	1	Finalizado
13	46	Diseñar interfaz para login de usuarios (administradores y administrativos)	Aldana	1	Finalizado
13	47	Desarrollar lógica para login de usuarios	Franco	2	Finalizado
13	59	Realizar pruebas de seguridad	Lara	2	Finalizado
13	61	Diseñar interfaces de menú para usuarios administradores y administrativos	Aldana	1	Finalizado
12	55	Desarrollar lógica para modificar un usuario administrativo	Santiago	2	Finalizado
12	56	Diseñar interfaces para modificar datos de un usuario administrativo con sus validaciones	Aldana	1	Finalizado
12	65	Desarrollar lógica para buscar usuarios administrativos	Santiago	2	Finalizado
12	57	Realizar pruebas de búsqueda y modificación de usuarios administrativos	Santiago	1	Finalizado

Tabla 12 – Estado del sprint backlog en daily meeting 4

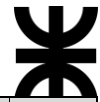
Nota sobre las daily meetings:

Si bien Scrum recomienda reuniones diarias, adaptamos la frecuencia a 1-2 reuniones semanales por restricciones de cursada. Solo documentamos las reuniones clave para el seguimiento, pero mantuvimos comunicación informal constante (grupo de WhatsApp) para resolver bloqueos rápidos. Esta flexibilidad permitió equilibrar la metodología con la disponibilidad del equipo.



REVISIÓN DEL SPRINT

Cada programador realizó la estimación del esfuerzo necesario para las tareas que le fueron asignadas. Esto se encuentra representado en la tabla, específicamente en la columna “**Estimado**”. Por otro lado, en la columna “**Real**” se visualiza el esfuerzo real demandado por cada tarea.



ID Historia	ID Tarea	Nombre tarea	Estimado	Real	
8	37	Diseñar interfaz para la búsqueda de licencias vigentes	1	1	
8	38	Implementar lógica de búsqueda de licencias vigentes por criterios	2	1	
8	39	Realizar pruebas de funcionalidad de búsqueda de licencias	2	1	
8	62	Diseñar interfaz de menú de licencias	1	15 min	
8	66	Refactorizar "buscarLicenciasVigentes".	1	30 min	
7	40	Desarrollar lógica para listar licencias expiradas	4	8	
7	41	Diseñar interfaz de listado de licencias expiradas	2	4	
7	42	Realizar pruebas de listado de licencias expiradas	1	30 min	
5	25	Diseñar interfaz de renovación de licencia	1	15 min	
5	26	Desarrollar lógica de renovación de licencias	2	1	
5	27	Realizar pruebas de renovación de licencias	2	1	
5	67	Refactorizar "renovarLicencia"	1	1	
6	43	Desarrollar la funcionalidad para solicitar copia de una licencia	2	2,5	
6	45	Realizar pruebas de emisión de copias de licencias	1	1	
6	68	Refactorizar "emitirCopiaLicencia"	1	1	
10	52	Diseñar interfaces para modificar un titular con sus validaciones	1	2	
10	54	Desarrollar lógica de modificación de datos de un titular	1	30 min	
10	63	Desarrollar lógica para buscar titulares	2	2	
10	58	Realizar pruebas de búsqueda y modificación de titulares	2	2	
11	48	Implementar lógica de registro de un usuario administrativo	2	1,5	
11	50	Diseñar interfaces para el registro de un usuario administrativo con sus validaciones	1	30 min	
11	49	Realizar pruebas de creación de usuarios administrativos	1	1	
13	46	Diseñar interfaz para login de usuarios (administradores y administrativos)	1	15 min	
13	47	Desarrollar lógica para login de usuarios	2	4	
13	59	Realizar pruebas de seguridad	2	1	
13	61	Diseñar interfaces de menú para usuarios administradores y administrativos	1	30 min	
12	55	Desarrollar lógica para modificar un usuario administrativo	2	1	
12	56	Diseñar interfaces para modificar datos de un usuario administrativo con sus validaciones	1	1	
12	65	Desarrollar lógica para buscar usuarios administrativos	2	2	
12	57	Realizar pruebas de búsqueda y modificación de usuarios administrativos	1	15 min	
			46	36	TOTALES

Tabla 13 – Esfuerzo estimado por cada tarea vs. esfuerzo real



Durante este sprint logramos completar la mayoría de las tareas propuestas, alcanzando 36 de los 46 puntos estimados inicialmente. En general, trabajamos con eficiencia y logramos cumplir con los objetivos, aunque algunas tareas requirieron más esfuerzo del previsto. Particularmente en el desarrollo del login, enfrentamos desafíos porque era la primera vez que usábamos herramientas para implementar seguridad, lo que nos llevó más tiempo de lo planeado, pero también nos permitió aprender mucho. También hubo tareas que resolvimos más rápido de lo esperado, gracias a la experiencia previa y a una buena organización. Además, incorporamos la refactorización, que nos ayudó a mejorar el código y facilitar el mantenimiento. En resumen, fue un sprint en el que consolidamos conocimientos y resolvimos funcionalidades esenciales.

GRÁFICO BURNDOWN

Para este sprint pudimos utilizar el gráfico generado por Jira:

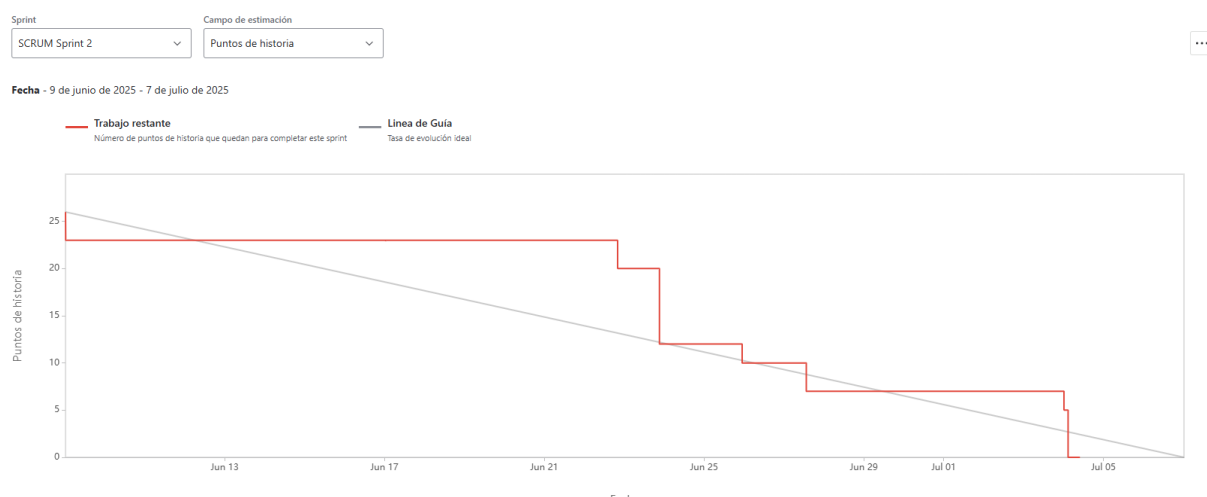


Gráfico burndown del sprint 2

Se puede notar que en este sprint tuvimos un arranque más lento de lo habitual, principalmente porque los integrantes del equipo estuvimos atravesando época de exámenes y entregas de otros trabajos prácticos en la facultad. Esto impactó directamente en nuestra disponibilidad y ritmo de trabajo en los primeros días, como se refleja en el gráfico: no se cerraron demasiadas historias de usuario hasta pasados los 10 días del sprint.

Sin embargo, más adelante pudimos organizarnos y retomar el foco en las tareas del sprint. Comenzamos a cerrar funcionalidades de forma más ágil, trabajando en bloques y manteniendo una comunicación fluida para integrar, refactorizar y testear de manera efectiva.

Más cerca del final tuvimos un salto fuerte en productividad, completando funcionalidades esenciales como la búsqueda de licencias y sus pruebas. Desde ahí, sostuvimos un ritmo más constante de entregas, cerrando tareas hasta llegar a completar el sprint en el último día.



CONCLUSIONES

A lo largo del proyecto aplicamos una metodología ágil basada en Scrum, que combinamos con prácticas esenciales de integración continua y estándares de codificación. Organizamos el trabajo en sprints, asignamos tareas específicas según los perfiles de cada integrante y mantuvimos una integración progresiva y constante del código. El uso de herramientas como Git para el control de versiones y la ejecución regular de pruebas (aunque en muchos casos manuales) nos permitió mantener la coherencia y estabilidad del sistema a medida que avanzábamos.

Uno de los mayores aciertos fue el establecimiento de estándares claros: definimos criterios comunes para la organización del código, el uso de DTOs, las validaciones y la estructura de formularios. Esto facilitó el trabajo en equipo, incluso cuando el desarrollo se realizaba en paralelo. También fue muy valioso haber incorporado tareas de refactorización, ya que ayudó a limpiar y optimizar el código luego de implementar funcionalidades críticas.

Sin embargo, no todo fue ideal. Al enfrentarnos al uso de tecnologías nuevas para el grupo, como Spring Security, surgieron dificultades técnicas por falta de experiencia previa. Tareas como el desarrollo del login nos tomaron más tiempo del estimado porque debimos aprender sobre la marcha. Este tipo de obstáculos evidenció la necesidad de mejorar nuestras habilidades en herramientas clave antes de integrarlas al proyecto.

Otra limitación fue la falta de automatización completa en la integración y validación. No llegamos a implementar herramientas como Jenkins o GitHub Actions, por lo que dependimos del esfuerzo manual para testear e integrar los cambios. Además, aunque seguimos principios de Scrum, nos alejamos un poco en la práctica: no realizamos dailys completas y usamos Jira de forma parcial. A pesar de eso, la comunicación constante por canales como WhatsApp y la responsabilidad compartida lograron compensar muchas de estas carencias.

En definitiva, fue un proyecto donde crecimos tanto técnica como grupalmente. Logramos cumplir con los objetivos funcionales, mantener la calidad del código y aplicar buenas prácticas que antes no dominábamos. En el futuro, fortalecer la automatización, profundizar en herramientas clave y mejorar el seguimiento ágil nos permitirá avanzar con mayor rapidez. La experiencia fue positiva y formativa, y sentó una base muy valiosa para proyectos futuros más complejos.